UNIVERSITATEA INTERNAȚIONALĂ DANUBIUS

LUCRARE DE LICENȚĂ

FACULTATEA DE INFORMATICĂ

DEZVOLTAREA UNEI APLICAȚII WEB MODERNE DE E-COMMERCE FOLOSIND TEHNOLOGII FULL-STACK

Coordonator științific: Prof. univ. dr. Radu Tonis Manea Bucea

Absolvent: Petrescu Cristian

Anul universitar: 2027

DECLARAȚIE DE ORIGINALITATE

Subsemnatul, Petrescu Cristian, declar pe propria răspundere că lucrarea de licență cu titlul "Dezvoltarea unei aplicații web moderne de e-commerce folosind tehnologii full-stack" este rezultatul propriei activități de cercetare și că nu am folosit alte surse decât cele menționate în bibliografie.

Declar că:

- Am respectat normele de etică academică și nu am comis plagiat
- Toate sursele utilizate sunt citate corespunzător
- Codul aplicației este original și dezvoltat personal
- Datele și rezultatele prezentate sunt reale și verificabile

Data: 5 ianuarie 2027

Semnătura: Petrescu Cristian

CUPRINS

PARTEA I - FUNDAMENTAREA TEORETICĂ 6

INTRODUCERE 7

Capitolul 1. INTRODUCERE ȘI OBIECTIVE 7

1.1. Istoricul comerțului electronic 7

1.2. Piața e-commerce în România și Europa 11

1.3. Tendințe actuale și viitoare 16

Capitolul 2. ANALIZA CERINȚELOR ȘI ARHITECTURA SISTEMULUI 18

2.1. Frontend frameworks (React, Next.js) 18

1.3. Structura lucrării 18

2.3. Baze de date (PostgreSQL, Prisma ORM) 19

2.2. Analiza cerințelor non-funcționale 30

2.3. Arhitectura generală a sistemului 31

2.4. Alegerea tehnologiilor 33

Capitolul 3. DESIGNUL INTERFEȚEI RESPONSIVE 37

3.1. Principiile design-ului modern 37

3.2. Implementarea responsive design 43

3.3. Experiența utilizatorului (UX) 52

3.4. Accesibilitatea aplicației 53

PARTEA II - IMPLEMENTAREA PRACTICĂ 55

CAPITOLUL 4. IMPLEMENTAREA ȘI SECURIZAREA APLICAȚIEI 56

4.1. Dezvoltarea frontend-ului 56

4.2. Dezvoltarea backend-ului 59

4.3. Integrarea bazei de date 66

4.4. Implementarea securității 69

4.7. Securitatea aplicației 73

5. TESTARE, REZULTATE ȘI CONCLUZII 77

5.1. Strategia de testare 77

5.2. Rezultatele testării 79

BIBLIOGRAFIE ȘI RESURSE 91

ANEXE 92

Anexa A 92

Anexa B: 96

ANEXA C: FRAGMENTE DE COD REPREZENTATIVE 97

Anexa D: Rezultate teste și metrici 139

Anexa E: Documentație tehnică 139

Anexa F: Codul sursă complet 139

FIGURE

Figura 3.1. Homepage-structură informațională 38

Figura 3.2. Product Page -structură informațională 39

Figura 3.3. Checkout -structură informațională 40

Figura 3.4 Mobile layout: 47

Figura 3.5 Mobile layout: 47

Figura 3.6 Desktop layout: 47

Figura 3.7 Vertical steps, one at a time 47

Figura 4.8. Produse 48

Figura 4.9, Produse 48

Figura 3.10. Produse 49

TABELE

Tabel 1.1. Evoluția pieței e-commerce în România (2020-2024) ... 8
Tabel 1.2. Comparație framework-uri frontend (2024) ... 12
Tabel 13. Comparație tehnologii backend Node.js (2024) ... 14
Tabel 1.4. Comparație soluții ORM/Query Builder pentru Node.js ... 15
Tabel 2.1. Prioritizare cerințe funcționale ... 29
Tabel 2.2. Cerințe de sistem pentru dezvoltare și rulare aplicație ... 30
Tabel 2.3. Variabile de mediu necesare pentru configurare aplicație ... 36
Tabel 3.1. Analiză comparativă platforme e-commerce ... 37
Tabel 3.2.Paleta de culori aplicație ... 41
Tabel 3.3.Sistemul tipografic ... 42
Tabel 3.4 Rezultate teste usability (n=15 participanți) ... 43
Tabel 3.5. Provocări responsive design și soluții implementate ... 44
Tabel 3.6. Breakpoint-uri și adaptări layout ... 45
Tabel 3.7. Adaptare product grid pe dispozitive ... 47
Tabel 3.8. Impact optimizări imagini ... 49
Tabel 4.1. Arhitectura în straturi a backend-ului ... 59
Tabel 4.2. Structura completă API endpoints ... 61
Tabel 4.3.Middleware-uri implementate ... 63
Tabel 4.4.Rate de schimb exemplu (față de RON) ... 72
Tabel 4.5. Exemple traduceri produse alimentare ... 73
Tabel 4.6. Exemple traduceri produse alimentare ... 74
Tabel 4.7.Exemple Protecție ... 74
Tabel 4.8.Exemple ... 76
Tabel 5.1. Rezultate acoperire teste unitare (Jest Coverage) ... 79
Tabel 5.2. Comparație acoperire teste cu benchmarks industrie ... 80
Tabel 5.3. Scor Lighthouse (Desktop și Mobile) ... 81
Tabel 5.4. Core Web Vitals - Rezultate detaliate ... 82
Tabel 5.5. Benchmark performanță vs competitori români ... 83
Tabel 5.6. Scenarii load testing implementate ... 84
Tabel 5.7. Rezultate detaliate load testing ... 84

PARTEA I - FUNDAMENTAREA TEORETICĂ

INTRODUCERE

Capitolul 1. INTRODUCERE ȘI OBIECTIVE

1.1. Istoricul comerțului electronic

1.1.1. Contextul global al e-commerce

În era digitală actuală, comerțul electronic a devenit o componentă esențială a economiei globale, transformând fundamental modul în care consumatorii interacționează cu brandurile și efectuează achiziții. Conform raportului eMarketer (2024), vânzările globale de e-commerce au depășit 6.3 trilioane USD în 2024, reprezentând aproximativ 22% din totalul vânzărilor retail la nivel mondial.

Evoluția comerțului electronic poate fi împărțită în mai multe etape distincte:

Perioada 1990-2000: Nașterea e-commerce

Primele platforme de comerț electronic au apărut odată cu popularizarea internetului. Amazon (1994) și eBay (1995) au fost pionieri în acest domeniu, demonstrând potențialul vânzărilor online. În această perioadă, provocările principale erau legate de încrederea consumatorilor, securitatea plăților și infrastructura tehnologică limitată.

Perioada 2000-2010: Maturizarea pieței

Această decadă a adus îmbunătățiri semnificative în tehnologiile web, apariția platformelor de plată securizate (PayPal, Stripe) și creșterea încrederii consumatorilor în achizițiile online. Conform studiilor Nielsen (2008), peste 85% dintre utilizatorii de internet efectuau achiziții online cel puțin ocazional.

Perioada 2010-2020: Revoluția mobilă

Proliferarea smartphone-urilor a transformat e-commerce-ul, introducând conceptul de m-commerce (mobile commerce). Conform Statista (2020), peste 50% din tranzacțiile e-commerce erau efectuate de pe dispozitive mobile. Această perioadă a văzut și apariția social commerce și integrarea rețelelor sociale în procesul de cumpărare.

Perioada 2020-prezent: Accelerarea digitală

Pandemia COVID-19 a accelerat dramatic adoptarea e-commerce, forțând atât consumatorii cât și afacerile să se adapteze rapid la mediul digital. Conform Adobe Analytics (2021), e-commerce-ul a crescut cu 42% în 2020, echivalentul a 4-6 ani de creștere organică concentrată într-un singur an.

1.1.2. Piața e-commerce în România

România a înregistrat una dintre cele mai rapide creșteri ale pieței e-commerce din Europa Centrală și de Est. Conform raportului GPeC (2024), piața românească de e-commerce a atins o valoare de aproximativ 8.5 miliarde EUR în 2024, cu o creștere anuală de 15-20%.

An	**Valoare piață (miliarde EUR**	**Creștere (%)**	**Nr. cumpărători online (milioane)**	**Valoare medie comandă (EUR)**
2020	5.2	+30%	8.5	245
2021	6.1	+17%	9.4	258
2022	6.9	+13%	10.5	270
2023	7.5	+9%	11.2	285
2024	8.5	+13%	12.1	295

Tabel 1.1. Evoluția pieței e-commerce în România (2020-2024)

(Sursa: GPeC E-commerce Report 2024, ARMO - Asociația Română a Magazinelor Online)

Caracteristici ale pieței românești:

Categorii de produse populare:

Conform studiului ARMO (2024), categoriile cu cea mai mare pondere în vânzările online din România sunt:

- Electronice și electrocasnice (28%)
- Fashion și încălțăminte (22%)
- Produse pentru casă și grădină (15%)
- Cosmetice și produse de îngrijire (12%)
- Cărți și papetărie (8%)
- Altele (15%)

Comportamentul consumatorilor români.

Studiile de piață (Nielsen, 2024) arată că consumatorul român online are următoarele caracteristici:

- Vârstă medie: 35 ani
- 52% femei, 48% bărbați
- 68% efectuează achiziții online lunar
- 35% efectuează achiziții săptămânal

- Timpul mediu petrecut pe un site e-commerce: 8.5 minute
- Rata de abandon a coșului: 72% (peste media europeană de 69%)

Provocări specifice pieței românești:

- Preferința pentru plata ramburs (45% din comenzi în 2024, în scădere de la 65% în 2020)
- Încrederea limitată în magazinele online noi
- Așteptări ridicate privind livrarea rapidă și gratuită
- Sensibilitate mare la preț și promoții

1.1.3. Tendințe actuale în e-commerce

Industria e-commerce continuă să evolueze rapid, fiind influențată de progresele tehnologice și schimbările în comportamentul consumatorilor. Următoarele tendințe definesc peisajul actual:

Personalizarea experienței:

Conform studiului Salesforce (2024), 73% dintre consumatori se așteaptă ca brandurile să înțeleagă nevoile și așteptările lor unice. Tehnologiile de machine learning și AI permit acum personalizarea recomandărilor de produse, a conținutului și a ofertelor în timp real.

Omnichannel și experiența unificată:

Consumatorii moderni se așteaptă la o experiență consistentă pe toate canalele - web, mobile, social media, magazine fizice. Conform Harvard Business Review (2023), clienții omnichannel cheltuiesc cu 30% mai mult decât cei care folosesc un singur canal.

Social commerce:

Integrarea funcționalităților de cumpărare direct în platformele social media (Instagram Shopping, Facebook Marketplace, TikTok Shop) devine din ce în ce mai importantă. eMarketer (2024) estimează că social commerce va reprezenta 20% din totalul vânzărilor e-commerce până în 2025.

Sustenabilitate și responsabilitate socială:

Conform IBM (2024), 70% dintre consumatori sunt dispuși să plătească o primă de 35% pentru branduri sustenabile și responsabile social. Acest lucru influențează atât produsele oferite cât și practicile de livrare și ambalare.

Voice commerce și IoT:

Asistentele vocale (Alexa, Google Assistant) și dispozitivele IoT creează noi oportunități pentru comerțul electronic. Juniper Research (2024) estimează că voice commerce va genera vânzări de peste 80 miliarde USD până în 2025.

1.1.4. Motivația dezvoltării proiectului

Dezvoltarea unei aplicații web moderne de e-commerce reprezintă o provocare tehnică complexă care necesită integrarea mai multor tehnologii avansate, respectarea standardelor de securitate și oferirea unei experiențe utilizator excepționale. Această lucrare își propune să demonstreze implementarea unei soluții complete de e-commerce folosind cele mai noi tehnologii full-stack disponibile în 2024-2025.

Justificarea alegerii temei:

Din perspectivă academică:

Proiectul permite aplicarea practică a conceptelor învățate pe parcursul studiilor universitare, incluzând:

- Ingineria software și arhitectura aplicațiilor
- Programare web avansată (frontend și backend)
- Managementul bazelor de date
- Securitatea aplicațiilor web
- Testarea și asigurarea calității software

Din perspectivă practică:

Competențele dobândite prin dezvoltarea acestui proiect sunt direct aplicabile în industrie, unde cererea pentru dezvoltatori full-stack cu experiență în e-commerce este în continuă creștere. Conform LinkedIn (2024), "Full-Stack Developer" este pe locul 3 în topul celor mai căutate poziții în tech.

Din perspectivă tehnologică:

Proiectul oferă oportunitatea de a lucra cu tehnologii moderne și relevante:

- React 19 și Next.js 16 pentru frontend modern
- Node.js și Fastify pentru backend performant
- PostgreSQL și Prisma pentru managementul datelor
- TypeScript pentru type safety
- Docker pentru containerizare și deployment

Provocări tehnice abordate:

Dezvoltarea acestei aplicații a implicat rezolvarea mai multor provocări tehnice complexe:

- Scalabilitate: Proiectarea unei arhitecturi care să poată gestiona creșterea numărului de utilizatori și produse fără degradarea performanței.
- Securitate: Implementarea măsurilor de securitate conform standardelor OWASP pentru protejarea datelor utilizatorilor și prevenirea atacurilor comune (XSS, CSRF, SQL Injection).

- Performanță: Optimizarea timpilor de încărcare și a experienței utilizator prin tehnici moderne (lazy loading, code splitting, caching, CDN).
- Accesibilitate: Asigurarea că aplicația poate fi utilizată de persoane cu dizabilități, conform standardelor WCAG 2.1.
- Internationalizare: Implementarea suportului pentru multiple limbi și monede pentru a permite extinderea pe piețe internaționale.

Motivația pentru alegerea acestui proiect derivă din necesitatea de a înțelege și implementa practic conceptele moderne de dezvoltare web, inclusiv:

- Arhitectura aplicațiilor full-stack moderne
- Managementul stării în aplicații complexe
- Securitatea aplicațiilor web la nivel enterprise
- Optimizarea performanței pentru experiență utilizator superioară
- Design responsive și accesibilitate conform standardelor internaționale
- Integrarea cu servicii externe (API-uri, sisteme de plată)
- Implementarea best practices în dezvoltarea software

1.2. Piața e-commerce în România și Europa

1.2.1. Evoluția tehnologiilor frontend

Ecosistemul tehnologiilor web a evoluat rapid în ultimii ani, oferind dezvoltatorilor instrumente din ce în ce mai sofisticate pentru crearea aplicațiilor moderne. Înțelegerea acestei evoluții este esențială pentru alegerea tehnologiilor potrivite pentru un proiect de e-commerce.

De la pagini statice la aplicații interactive:

Generația 1 (1990-2005): HTML static și CSS

Primele site-uri web erau compuse din pagini HTML statice cu styling CSS minimal. Interactivitatea era limitată, iar actualizarea conținutului necesita modificarea manuală a fișierelor HTML. Această abordare era suficientă pentru site-uri informaționale simple, dar inadecvată pentru aplicații complexe.

Generația 2 (2005-2010): AJAX și Web 2.0

Introducerea AJAX (Asynchronous JavaScript and XML) a permis actualizarea parțială a paginilor fără reîncărcare completă, îmbunătățind semnificativ experiența utilizator. Biblioteci precum jQuery au simplificat manipularea DOM-ului și gestionarea evenimentelor. Totuși, aplicațiile complexe deveneau dificil de menținut din cauza lipsei unei structuri clare.

Generația 3 (2010-2015): Framework-uri MVC

Apariția framework-urilor precum AngularJS, Backbone.js și Ember.js a introdus concepte de arhitectură (MVC/MVVM) în dezvoltarea frontend. Acestea ofereau structură și organizare, dar adesea cu o curbă de învățare abruptă și performanță suboptimă pentru aplicații mari.

Generația 4 (2015-prezent): Component-based și Virtual DOM

React (2013), Vue.js (2014) și Angular 2+ (2016) au revoluționat dezvoltarea frontend prin introducerea arhitecturii bazate pe componente și concepte precum Virtual DOM. Această abordare permite crearea de aplicații complexe, performante și ușor de menținut.

1.2.2. Comparație framework-uri frontend moderne

Pentru a justifica alegerea tehnologiilor folosite în acest proiect, este esențială o analiză comparativă a principalelor framework-uri disponibile în 2024.

Criteriu	**React + Next.js**	**Vue.js + Nuxt**	**Angular**	**Svelte/SvelteKit**
Performanță	Excelentă	Foarte bună	Bună	Excelentă
Curba învățare	Medie	Ușoară	Dificilă	Ușoară
Ecosistem	Foarte mare	Mare	Mare	În creștere
TypeScript	Excelent	Bun	Nativ	Bun
SSR/SSG	Da (Next.js)	Da (Nuxt)	Limitat	Da (SvelteKit)
Comunitate	220k+ stars	210k+ stars	95k+ stars	75k+ stars
Companii	Meta, Netflix, Airbnb	Alibaba, GitLab	Alibaba, GitLab	NY Times, Apple
Bundle size	Mediu	Mic	Mare	Foarte mic
Adopție piață	42%	18%	20%	5%

Tabel 1.2. Comparație framework-uri frontend (2024)

(Sursa: State of JS 2024, Stack Overflow Developer Survey 2024, npm trends)

Justificarea alegerii React + Next.js:

Ecosistem matur și bogat:

React beneficiază de cel mai mare ecosistem de biblioteci și componente reutilizabile. npm registry conține peste 150,000 de pachete compatibile cu React, acoperind practic orice funcționalitate necesară într-o aplicație modernă.

Performanță și optimizări:

Next.js 16 introduce optimizări semnificative:

- Server Components pentru reducerea JavaScript-ului trimis clientului
- Automatic Image Optimization pentru încărcare rapidă a imaginilor
- Incremental Static Regeneration pentru conținut dinamic
- Edge Runtime pentru latență minimă

Suport pentru TypeScript:

Integrarea nativă cu TypeScript oferă type safety complet, reducând bug-urile și îmbunătățind experiența de development prin IntelliSense și refactoring automat.

SEO și performanță:

Next.js oferă Server-Side Rendering (SSR) și Static Site Generation (SSG) out-of-the-box, esențiale pentru SEO și performanță în aplicațiile e-commerce unde timpul de încărcare influențează direct conversiile.

Adopție în industrie:

Conform Stack Overflow Survey (2024), React este cel mai folosit framework frontend, cu 42% adopție, urmat de Vue.js (18%) și Angular (20%). Această adopție largă asigură disponibilitatea resurselor, documentației și a dezvoltatorilor cu experiență.

1.2.3. Tehnologii backend moderne

Evoluția arhitecturilor backend:

Arhitecturi monolitice tradiționale:

Aplicațiile tradiționale erau construite ca un singur bloc monolitic, unde toate funcționalitățile erau strâns cuplate. Această abordare simplifica deployment-ul inițial dar devenea problematică la scalare și mentenanță.

Arhitecturi SOA (Service-Oriented Architecture):

SOA a introdus conceptul de servicii independente care comunică prin protocoale standardizate (SOAP, REST). Această abordare îmbunătățea modularitatea dar adesea cu overhead semnificativ.

Arhitecturi microservicii:

Microserviciile împart aplicația în servicii mici, independente, fiecare responsabil pentru o funcționalitate specifică. Această abordare oferă scalabilitate și flexibilitate maximă dar introduce complexitate în orchestrare și comunicare.

Arhitecturi serverless:

Funcțiile serverless (AWS Lambda, Azure Functions) permit rularea codului fără gestionarea infrastructurii. Această abordare reduce costurile și complexitatea operațională pentru anumite tipuri de aplicații.

Criteriu	**Fastify**	**Express**	**NestJS**	**Koa**
Performanță (req/s)	76,000	38,000	42,000	50,000
Overhead	Minim	Mediu	Mediu-Mare	Mic
Validare schema	Nativă (JSON Schema)	Plugin	Class-validator	Plugin
Documentație	Excelentă	Foarte bună	Excelentă	Bună
Ecosistem	În creștere	Foarte mare	Mare	Mediu
Curba învățare	Medie	Ușoară	Medie-Dificilă	Medie
Adopție	15%	55%	18%	8%

Tabel 1.3. Comparație tehnologii backend Node.js (2024)

(Sursa: Fastify Benchmarks 2024, npm trends, State of JS 2024)

Justificarea alegerii Fastify:

Performanță superioară:

Fastify este unul dintre cele mai rapide framework-uri Node.js, capabil să proceseze până la 76,000 request-uri pe secundă (conform benchmarks oficiale), de două ori mai rapid decât Express. Pentru o aplicație e-commerce unde timpul de răspuns influențează direct experiența utilizatorului și conversiile, această performanță este crucială.

Validare nativă cu JSON Schema:

Fastify include validare nativă bazată pe JSON Schema, eliminând necesitatea bibliotecilor externe și asigurând validarea rapidă și type-safe a datelor de intrare. Acest lucru reduce semnificativ riscul de erori și vulnerabilități de securitate.

Suport TypeScript de primă clasă:

Fastify oferă type definitions complete și actualizate, permițând dezvoltarea type-safe end-to-end. Acest lucru reduce bug-urile și îmbunătățește productivitatea dezvoltatorilor.

Arhitectură bazată pe plugin-uri:

Sistemul de plugin-uri al Fastify permite extinderea funcționalității într-un mod modular și mentenabil, facilitând separarea responsabilităților și reutilizarea codului.

1.2.4. Managementul bazelor de date

Evoluția tehnologiilor de baze de date:

Baze de date relaționale (SQL):

PostgreSQL, MySQL, SQL Server oferă consistență ACID, relații complexe și query-uri puternice. Sunt ideale pentru aplicații unde integritatea datelor este critică, cum ar fi sistemele e-commerce.

Baze de date NoSQL:

MongoDB, Redis, Cassandra oferă scalabilitate orizontală și flexibilitate în schema datelor. Sunt potrivite pentru aplicații cu volume mari de date nestructurate.

Baze de date NewSQL:

CockroachDB, Google Spanner combină avantajele SQL (consistență ACID) cu scalabilitatea NoSQL.

Criteriu	Prisma	TypeORM	Sequelize	Knex.js
TypeScript	Nativ	Nativ	Bun	Bun
Type Safety	Excelentă	Bun	Mediu	Slab
Performanță	Excelentă	Bun	Bună	Excelentă
Migrații	Automate	Manuale	Manuale	Manuale
Developer Experience	Excelentă	Bun	Mediu	Mediu
Relații	Intuitive	Complexe	Mediu	Manuale
Adopție	În creștere rapidă	Stabilă	Largă	Mediu

Tabel 1.4. Comparație soluții ORM/Query Builder pentru Node.js

(Sursa: npm trends 2024, GitHub stars, State of JS 2024)

Justificarea alegerii PostgreSQL + Prisma:

PostgreSQL - Robustețe și funcționalități:

- Conformitate ACID completă pentru tranzacții sigure
- Suport pentru JSON/JSONB pentru flexibilitate

- Full-text search nativ
- Performanță excelentă pentru query-uri complexe
- Extensibilitate prin funcții și proceduri stocate
- Comunitate activă și documentație excelentă

Prisma - Developer Experience superior:

- Type safety complet cu TypeScript
- Generare automată de tipuri din schema bazei de date
- Migrații automate și sigure
- Prisma Studio pentru vizualizare și editare date
- Query-uri intuitive și type-safe
- Performanță optimizată prin query batching și caching

1.2.5. Tendințe și tehnologii emergente

Inteligență Artificială și Machine Learning:

Integrarea AI în aplicațiile web devine din ce în ce mai accesibilă prin API-uri precum OpenAI GPT-4, Google Gemini. Aplicațiile e-commerce pot beneficia de:

- Recomandări personalizate de produse
- Chatbots inteligenți pentru suport clienți
- Analiză sentiment pentru recenzii
- Optimizare prețuri dinamică

Edge Computing:

Platforme precum Cloudflare Workers, Vercel Edge Functions permit rularea codului cât mai aproape de utilizatori, reducând latența și îmbunătățind performanța globală.

WebAssembly (WASM):

Permite rularea codului compilat în browser cu performanță aproape nativă, deschizând posibilități pentru aplicații web complexe care anterior necesitau aplicații native.

Progressive Web Apps (PWA):

PWA-urile combină avantajele aplicațiilor web (accesibilitate, actualizări instant) cu cele ale aplicațiilor native (funcționare offline, notificări push, instalare pe device).

Jamstack Architecture:

Arhitectura Jamstack (JavaScript, APIs, Markup) separă frontend-ul de backend, oferind performanță, securitate și scalabilitate superioare prin pre-rendering și servire statică.

1.3. Tendințe actuale și viitoare

Personalizarea experienței:

Conform studiului Salesforce (2024), 73% dintre consumatori se așteaptă ca brandurile să înțeleagă nevoile și așteptările lor unice. Tehnologiile de machine learning și AI permit acum personalizarea recomandărilor de produse, a conținutului și a ofertelor în timp real.

Omnichannel și experiența unificată:

Consumatorii moderni se așteaptă la o experiență consistentă pe toate canalele - web, mobile, social media, magazine fizice. Conform Harvard Business Review (2023), clienții omnichannel cheltuiesc cu 30% mai mult decât cei care folosesc un singur canal.

Social commerce:

Integrarea funcționalităților de cumpărare direct în platformele social media (Instagram Shopping, Facebook Marketplace, TikTok Shop) devine din ce în ce mai importantă. eMarketer (2024) estimează că social commerce va reprezenta 20% din totalul vânzărilor e-commerce până în 2025.

Sustenabilitate și responsabilitate socială:

Conform IBM (2024), 70% dintre consumatori sunt dispuși să plătească o primă de 35% pentru branduri sustenabile și responsabile social. Acest lucru influențează atât produsele oferite cât și practicile de livrare și ambalare.

Voice commerce și IoT:

Asistentele vocale (Alexa, Google Assistant) și dispozitivele IoT creează noi oportunități pentru comerțul electronic. Juniper Research (2024) estimează că voice commerce va genera vânzări de peste 80 miliarde USD până în 2025.

Capitolul 2. ANALIZA CERINȚELOR ȘI ARHITECTURA SISTEMULUI

2.1. Frontend frameworks (React, Next.js)

Obiectivul general:

Dezvoltarea unei aplicații web complete de e-commerce care să demonstreze utilizarea tehnologiilor moderne full-stack și să ofere o experiență utilizator de înaltă calitate.

Obiective specifice:

1. Obiective tehnice

- Implementarea unei arhitecturi scalabile folosind React.js 19.2.0 și Next.js 16.0.1
- Dezvoltarea unui API robust cu Fastify 5.6.2 și integrarea cu PostgreSQL
- Utilizarea Prisma 6.19.0 pentru managementul bazei de date
- Implementarea autentificării și autorizării cu JWT
- Crearea unui design responsive cu Tailwind CSS 4

2. Obiective funcționale:

- Dezvoltarea unui sistem complet de gestionare produse
- Implementarea funcționalității de coș de cumpărături
- Crearea sistemului de comenzi și facturare
- Dezvoltarea panoului de administrare
- Implementarea sistemului de recenzii și favorite

3. Obiective de calitate:

- Asigurarea securității aplicației
- Optimizarea performanței pentru încărcare rapidă
- Respectarea principiilor de accesibilitate web
- Implementarea testării automate
- Documentarea completă a codului

1.3. Structura lucrării

Această lucrare este organizată în șase capitole principale:

Capitolul 1 prezintă contextul, motivația și obiectivele proiectului, oferind o perspectivă generală asupra problemei abordate.

Capitolul 2 se concentrează pe analiza cerințelor și proiectarea arhitecturii sistemului, incluzând alegerea tehnologiilor și justificarea deciziilor de design.

Capitolul 3 detaliază procesul de design al interfeței utilizator, cu accent pe principiile responsive design și experiența utilizatorului.

Capitolul 4 prezintă implementarea practică a aplicației, atât pentru frontend cât și pentru backend, incluzând aspectele de securitate.

Capitolul 5 discută strategia de testare, rezultatele obținute și concluziile proiectului.

Capitolul 6 conține bibliografia și resursele utilizate în dezvoltarea proiectului.

2.3. Baze de date (PostgreSQL, Prisma ORM)

Evoluția tehnologiilor de baze de date:

Baze de date relaționale (SQL):

PostgreSQL, MySQL, SQL Server oferă consistență ACID, relații complexe și query-uri puternice. Sunt ideale pentru aplicații unde integritatea datelor este critică, cum ar fi sistemele e-commerce.

Baze de date NoSQL:

MongoDB, Redis, Cassandra oferă scalabilitate orizontală și flexibilitate în schema datelor. Sunt potrivite pentru aplicații cu volume mari de date nestructurate.

Baze de date NewSQL:

CockroachDB, Google Spanner combină avantajele SQL (consistență ACID) cu scalabilitatea NoSQL.

Dezvoltarea unei aplicații de e-commerce necesită o analiză detaliată a cerințelor funcționale pentru a asigura că toate aspectele esențiale ale unui sistem de comerț electronic sunt acoperite. Această analiză a fost realizată prin studierea aplicațiilor existente pe piață, consultarea best practices din industrie și identificarea nevoilor utilizatorilor țintă.

2.1.1. Metodologia de analiză a cerințelor

Procesul de analiză a cerințelor a urmat metodologia Agile, combinând tehnici precum:

- User Stories: Descrierea funcționalităților din perspectiva utilizatorului final, urmând formatul "Ca [rol], vreau [funcționalitate], pentru a [beneficiu]".
- Use Cases: Scenarii detaliate care descriu interacțiunea utilizatorului cu sistemul pentru atingerea unui obiectiv specific.
- Personas: Profiluri fictive ale utilizatorilor tipici, bazate pe cercetarea pieței și analiza comportamentului consumatorilor români online.
- MoSCoW Prioritization: Clasificarea cerințelor în Must have, Should have, Could have, Won't have pentru prioritizare eficientă.

2.1.2. Personas - Profiluri utilizatori țintă

Pentru a înțelege mai bine nevoile utilizatorilor, au fost create trei personas reprezentative:

Persona 1: Maria, 32 ani - Consumatorul frecvent

- Ocupație: Marketing Manager
- Experiență tehnică:Medie-Avansată
- Comportament: Efectuează 3-4 comenzi online lunar, preferă plata cu cardul, apreciază livrarea rapidă
- Nevoi: Interfață intuitivă, proces de checkout rapid, tracking comenzi în timp real
- Frustrări: Procese complicate de checkout, lipsa informațiilor despre produse, timpi de încărcare lungi

Persona 2: Ion, 45 ani - Consumatorul ocazional

- Ocupație: Inginer
- Experiență tehnică: Medie
- Comportament: Cumpără online 1-2 ori pe lună, preferă ramburs, compară prețurile pe multiple site-uri
- Nevoi: Informații detaliate despre produse, recenzii verificate, opțiuni multiple de plată
- Frustrări: Costuri ascunse, lipsa transparenței în prețuri, dificultăți în găsirea produselor

Persona 3: Admin Andrei, 28 ani - Administratorul magazinului

- Ocupație: E-commerce Manager
- Experiență tehnică: Avansată
- Comportament: Gestionează zilnic produse, comenzi, utilizatori
- Nevoi: Dashboard intuitiv, rapoarte detaliate, gestionare eficientă a stocurilor
- Frustrări: Interfețe complicate, lipsa automatizărilor, raportare insuficientă

2.1.3. User Stories - Funcționalități din perspectiva utilizatorului

Modul Autentificare și Gestionare Cont:

US-001: Ca utilizator nou, vreau să mă pot înregistra rapid folosind email-ul, pentru a putea efectua comenzi.

-Criterii de acceptare:

- Formular cu câmpuri: nume, email, parolă, confirmare parolă
- Validare în timp real (email valid, parolă min. 8 caractere)
- Mesaj de confirmare prin email
- Redirect automat către dashboard după înregistrare

- Prioritate: Must have

US-002: Ca utilizator înregistrat, vreau să mă pot autentifica securizat, pentru a accesa contul meu.

- Criterii de acceptare:

- Autentificare cu email și parolă
- Opțiune "Ține-mă minte" pentru sesiuni persistente
- Link "Am uitat parola" funcțional
- Mesaje de eroare clare pentru credențiale incorecte
- Prioritate:Must have

US-003:Ca utilizator autentificat, vreau să îmi pot gestiona profilul, pentru a actualiza informațiile personale.

- Criterii de acceptare:
- Editare nume, email, telefon, adresă
- Schimbare parolă cu validare parolă curentă
- Vizualizare istoric comenzi
- Gestionare adrese de livrare multiple
- Prioritate: Must have

Modul Catalog Produse:

US-004: Ca vizitator, vreau să pot naviga prin categorii de produse, pentru a găsi ușor ce caut.

- Criterii de acceptare:
- Meniu categorii vizibil și intuitiv
- Subcategorii pentru organizare ierarhică
- Breadcrumbs pentru navigare ușoară
- Număr produse afișat pentru fiecare categorie
- Prioritate: Must have

US-005: Ca utilizator, vreau să pot căuta produse după nume sau descriere, pentru a găsi rapid produsul dorit.

- Criterii de acceptare:
- Bară de căutare vizibilă în header
- Sugestii automate în timp ce tastez (autocomplete)
- Rezultate relevante ordonate după relevanță
- Filtrare și sortare rezultate căutare
- Prioritate: Must have

US-006: Ca utilizator, vreau să pot filtra produsele după preț, categorie, disponibilitate, pentru a restrânge opțiunile.

-Criterii de acceptare:
- Filtre multiple aplicabile simultan

- Slider pentru interval preț
- Checkbox-uri pentru categorii și disponibilitate
- Număr rezultate actualizat în timp real
- Opțiune "Resetare filtre"

- Prioritate: Should have

US-007: Ca utilizator, vreau să pot sorta produsele după diferite criterii, pentru a găsi cele mai potrivite opțiuni.

- Criterii de acceptare:

- Sortare după: preț (crescător/descrescător), nume, dată adăugare, popularitate
- Dropdown vizibil pentru selecție sortare
- Aplicare instantanee fără reîncărcare pagină

- Prioritate: Should have

US-008: Ca utilizator, vreau să văd detalii complete despre un produs, pentru a lua o decizie informată de cumpărare.

- Criterii de acceptare:

- Imagini multiple cu zoom
- Descriere detaliată
- Preț clar afișat (cu reducere dacă există)
- Disponibilitate stoc
- Specificații tehnice în tabel
- Recenzii alți utilizatori
- Produse similare/recomandate

- Prioritate: Must have

Modul Coș de Cumpărături:

US-009: Ca utilizator, vreau să pot adăuga produse în coș, pentru a le cumpăra mai târziu.

- Criterii de acceptare:

- Buton "Adaugă în coș" vizibil pe fiecare produs
- Feedback vizual la adăugare (animație, notificare)
- Actualizare număr produse în icon coș din header
- Persistență coș între sesiuni

- Prioritate: Must have

US-010:Ca utilizator, vreau să pot modifica cantitatea produselor din coș, pentru a ajusta comanda.

- Criterii de acceptare:

- Butoane +/- pentru ajustare cantitate
- Input manual pentru cantitate
- Validare cantitate maximă (stoc disponibil)
- Actualizare automată total

- Prioritate: Must have

US-011: Ca utilizator, vreau să pot șterge produse din coș, pentru a elimina articolele nedorite.

- Criterii de acceptare:

- Buton "Șterge" pentru fiecare produs
- Confirmare înainte de ștergere
- Opțiune "Golește coș" pentru ștergere totală
- Mesaj când coșul este gol

- Prioritate:Must have

US-012: Ca utilizator, vreau să văd totalul comenzii actualizat în timp real, pentru a ști cât voi plăti.

- Criterii de acceptare:

- Subtotal produse
- Cost livrare (dacă aplicabil)
- Reduceri aplicate (vouchere, promoții)
- Total final evidențiat

- Prioritate: Must have

Modul Plasare Comandă:

US-013: Ca utilizator autentificat, vreau să pot plasa o comandă rapid, pentru a finaliza achiziția.

- Criterii de acceptare:

- Formular checkout cu date pre-completate din profil
- Selecție adresă livrare (existentă sau nouă)
- Selecție metodă plată (card, ramburs, transfer)
- Selecție metodă livrare (curier, easybox, ridicare)
- Câmp pentru observații
- Rezumat comandă înainte de confirmare

- Prioritate: Must have

US-014: Ca utilizator, vreau să primesc confirmare comandă prin email, pentru a avea dovada achiziției.

- Criterii de acceptare:

- Email automat după plasare comandă
- Detalii complete comandă (produse, prețuri, adresă)
- Număr comandă unic
- Link tracking comandă
- Informații contact suport

- Prioritate: Must have

US-015: Ca utilizator, vreau să pot urmări statusul comenzii, pentru a ști când va ajunge.

- Criterii de acceptare:

- Pagină "Comenzile mele" cu istoric
- Status clar pentru fiecare comandă (procesare, expediată, livrată)
- Timeline vizual cu etape
- Notificări email la schimbare status

- Prioritate: Should have

Modul Recenzii și Evaluări:

US-016: Ca utilizator care a cumpărat un produs, vreau să pot lăsa o recenzie, pentru a ajuta alți cumpărători.

- Criterii de acceptare:

- Formular recenzie cu rating (1-5 stele) și text
- Validare: doar utilizatori care au cumpărat produsul
- Opțiune adăugare poze
- Moderare recenzii înainte de publicare

- Prioritate: Should have

US-017: Ca utilizator, vreau să văd recenziile altor cumpărători, pentru a evalua calitatea produsului.

- Criterii de acceptare:

- Rating mediu afișat vizibil
- Număr total recenzii
- Filtrare recenzii după rating
- Sortare după dată sau utilitate
- Opțiune "Recenzie utilă" (like)

- Prioritate: Should have

Modul Administrare (Admin):

US-018: Ca administrator, vreau să pot gestiona produsele, pentru a menține catalogul actualizat.

- Criterii de acceptare:

- CRUD complet produse (Create, Read, Update, Delete)
- Upload imagini multiple
- Gestionare categorii și subcategorii
- Setare prețuri, reduceri, stocuri
- Import/export produse CSV

- Prioritate:Must have

US-019: Ca administrator, vreau să pot gestiona comenzile, pentru a procesa vânzările eficient.

- Criterii de acceptare:

- Listă comenzi cu filtrare și sortare
- Schimbare status comandă
- Vizualizare detalii complete comandă
- Generare factură PDF
- Statistici vânzări

- Prioritate: Must have

US-020: Ca administrator, vreau să văd rapoarte și statistici, pentru a analiza performanța magazinului.

- Criterii de acceptare:

- Dashboard cu metrici cheie (vânzări, comenzi, utilizatori)
- Grafice evoluție în timp
- Top produse vândute
- Rapoarte exportabile (PDF, Excel)

- Prioritate: Should have

2.1.4. Scenarii de utilizare detaliate

Scenariul 1: Cumpărare produs de către utilizator nou

Actor principal:Maria (utilizator nou)

Precondiții: Maria accesează site-ul pentru prima dată

Flux principal:

1. Maria navighează pe homepage și vede produsele promovate

2. Folosește bara de căutare pentru a găsi "branză"

3. Aplică filtre: preț 40-50 RON, disponibil în stoc

4. Sortează rezultatele după preț crescător

5. Accesează pagina produsului "Brnză de vacă"

6. Citește descrierea, specificațiile și recenziile

7. Adaugă produsul în coș

8. Accesează coșul și verifică totalul

9. Inițiază procesul de checkout

10. Sistemul solicită autentificare/înregistrare

11. Maria se înregistrează cu email și parolă

12. Completează adresa de livrare

13. Selectează plata cu cardul și livrare curier

14. Confirmă comanda

15. Primește email de confirmare cu număr comandă

- Flux alternativ 1: La pasul 13, Maria alege plata ramburs
- Flux alternativ 2: La pasul 11, Maria se autentifică cu cont existent
- Flux excepție: La pasul 14, plata cu cardul eșuează - sistemul afișează eroare și permite reîncercare

Postcondiții:

- Comanda este înregistrată în sistem

- Stocul produsului este decrementat

- Maria primește email de confirmare

- Administratorul vede comanda nouă în dashboard

Scenariul 2: Administrator procesează o comandă

Actor principal: Admin

Precondiții: Există comenzi noi în sistem

Flux principal:

1. Admin se autentifică în panoul de administrare

2. Accesează secțiunea "Comenzi"

3. Vede lista comenzilor cu status "Nouă"

4. Selectează o comandă pentru procesare

5. Verifică detaliile comenzii (produse, adresă, plată)

6. Verifică disponibilitatea produselor în stoc

7. Schimbă statusul comenzii în "În procesare"

8. Generează factura PDF

9. Pregătește produsele pentru expediere

10. Introduce AWB-ul curierului

11. Schimbă statusul în "Expediată"

12. Sistemul trimite email automat clientului cu AWB

- Flux alternativ: La pasul 6, un produs nu este în stoc - Admin contactează clientul pentru înlocuire/anulare
- Flux excepție: La pasul 8, generarea facturii eșuează - sistemul înregistrează eroarea și notifică administratorul

Postcondiții:

- Comanda are status "Expediată"
- Clientul primește email cu AWB
- Factura este generată și salvată
- Stocurile sunt actualizate

2.1.5. Cerințe funcționale prioritizate (MoSCoW)

ID	Cerință	Prioritate	Complexitate	Dependențe
CF-001	Autentificare utilizatori	Must have	Medie	-
CF-002	Înregistrare utilizatori	Must have	Medie	-
CF-003	Gestionare profil	Must have	Mică	CF-001
CF-004	Catalog produse	Must have	Mare	-
CF-005	Căutare produse	Should have	Medie	CF-004
CF-006	Filtrare produse	Should have	Mică	CF-004
CF-007	Sortare produse	Should have	Mică	CF-004
CF-008	Detalii produs	Must have	Medie	CF-004
CF-009	Coș cumpărături	Must have	Mare	CF-001

CF-010	Modificare cantitate coș	Must have	Mică	CF-009
CF-011	Plasare comandă	Must have	Mare	CF-001, CF-009
CF-012	Tracking comandă	Should have	Medie	CF-011
CF-013	Recenzii produse	Should have	Medie	CF-001, CF-011
CF-014	Favorite produse	Could have	Mică	CF-001
CF-015	Wishlist	Could have	Mică	CF-001
CF-016	Comparare produse	Could have	Medie	CF-004
CF-017	Admin - CRUD produse	Must have	Mare	CF-001
CF-018	Admin Gestionare comenzi	Must have	Mare	CF-001, CF-011
CF-019	Admin Rapoarte	Should have	Mare	CF-018
CF-020	Admin Gestionare utilizatori	Should have	Medie	CF-001
CF-021	Sistem vouchere	Should have	Medie	CF-011
CF-022	Conversie valutară	Should have	Medie	CF-004
CF-023	Traduceri multiple limbi	Should have	Mare	-

CF-024	Notificări push	Could have	Medie	CF-001
CF-025	Chat suport live	Could have	Mare	CF-001

Tabel 2.1. Prioritizare cerințe funcționale

(Sursa: proprie, bazat pe analiza cerințelor și prioritizare MoSCoW)

- Gestionarea sesiunilor utilizatorilor

2.1.2. Catalogul de produse

Funcționalitățile necesare includ:

- Afișarea produselor în format grid responsive
- Organizarea produselor pe categorii
- Funcționalitatea de căutare avansată
- Filtrarea produselor după diverse criterii
- Afișarea detaliilor complete ale produselor
- Gestionarea imaginilor produselor
- Sistemul de recenzii și rating-uri

2.1.3. Coșul de cumpărături

Sistemul de coș trebuie să ofere:

- Adăugarea produselor în coș
- Modificarea cantităților
- Ștergerea produselor din coș
- Calcularea automată a totalurilor
- Persistența coșului între sesiuni
- Validarea disponibilității stocului

2.1.4. Procesul de comandă

Funcționalitățile de comandă includ:

- Procesul de checkout pas cu pas
- Selecția metodelor de plată
- Alegerea opțiunilor de livrare
- Validarea datelor de comandă
- Generarea automată a facturilor
- Tracking-ul statusului comenzilor

2.1.5. Panoul de administrare

Administratorii trebuie să poată:

- Gestiona utilizatorii sistemului
- Adăuga, edita și șterge produse
- Gestiona categoriile de produse
- Procesa comenzile clienților
- Genera rapoarte de vânzări
- Gestiona voucherele și ofertele

2.2. Analiza cerințelor non-funcționale

Cerințele non-funcționale sunt la fel de importante ca cele funcționale, determinând calitatea generală a aplicației și experiența utilizatorului.

Componentă	Versiune Minimă	Versiune Recomandată	Observații
Node.js	18.x	20.x LTS	Runtime JavaScript pentru backend
PostgreSQ	14.x	16.x	Bază de date relațională
Npm	8.x	10.x	Package manager
Browser	Chrome 90+	Chrome 120+	Suport ES2022 și module moderne
RAM	4 GB	8 GB	Pentru mediu development
Disk Space	2 GB	5 GB	Include node_modules și dependencies

Tabel 2.2. Cerințe de sistem pentru dezvoltare și rulare aplicație

(Sursa: proprie)

2.2.1. Performanța

Aplicația trebuie să respecte următoarele criterii de performanță:

- Timpul de încărcare inițială sub 3 secunde
- Timpul de răspuns pentru operațiile CRUD sub 500ms
- Suportul pentru minimum 100 utilizatori concurenți
- Optimizarea imaginilor pentru încărcare rapidă
- Implementarea cache-ului pentru datele frecvent accesate

2.2.2. Securitatea

Măsurile de securitate implementate includ:

- Criptarea parolelor folosind algoritmi siguri
- Protecția împotriva atacurilor XSS și CSRF
- Validarea și sanitizarea tuturor input-urilor
- Implementarea rate limiting-ului
- Utilizarea HTTPS pentru toate comunicațiile
- Gestionarea securizată a token-urilor JWT

2.2.3. Scalabilitatea

Arhitectura trebuie să permită:

- Scalarea orizontală a serviciilor
- Separarea responsabilităților între componente
- Utilizarea unui ORM pentru abstractizarea bazei de date
- Implementarea unui sistem de cache distribuit
- Suportul pentru load balancing

2.2.4. Usabilitatea

Interfața trebuie să fie:

- Intuitivă și ușor de navigat
- Responsive pentru toate tipurile de dispozitive
- Accesibilă pentru utilizatorii cu dizabilități
- Consistentă în design și comportament
- Optimizată pentru conversii

2.2.5. Compatibilitatea

Aplicația trebuie să funcționeze pe:

- Toate browserele moderne (Chrome, Firefox, Safari, Edge)
- Dispozitive mobile (iOS, Android)
- Tablete și desktop-uri
- Diferite rezoluții de ecran
- Conexiuni internet lente

2.3. Arhitectura generală a sistemului

Arhitectura aplicației a fost proiectată urmând principiile moderne de dezvoltare software, cu accent pe separarea responsabilităților, scalabilitate și mentenabilitate.

2.3.1. Arhitectura de nivel înalt

Sistemul este organizat într-o arhitectură în trei niveluri:

1.Nivelul de prezentare (Frontend)

- Aplicația React.js cu Next.js

- Componente reutilizabile
- State management cu Context API
- Styling cu Tailwind CSS

2.Nivelul de logică de business (Backend)

- API REST dezvoltat cu Fastify
- Middleware pentru autentificare și validare
- Servicii pentru logica de business
- Gestionarea erorilor centralizată

3.Nivelul de date (Database)

- Baza de date PostgreSQL
- ORM Prisma pentru abstractizare
- Migrații automate
- Indexare optimizată

2.3.2. Arhitectura frontend-ului

Frontend-ul utilizează arhitectura componentelor React cu următoarea organizare:

```
frontend/
├── app/                # App Router (Next.js 13+)
│   ├── (dashboard)/      # Grouped routes
│   ├── globals.css       # Stiluri globale
│   └── layout.tsx        # Layout principal
├── components/           # Componente reutilizabile
│   ├── ui/             # Componente UI de bază
│   ├── forms/          # Componente pentru formulare
│   └── admin/          # Componente pentru admin
├── lib/                # Utilitare și configurări
│   ├── api-client.ts     # Client pentru API
│   ├── auth-context.tsx  # Context pentru autentificare
│   └── utils.ts          # Funcții utilitare
└── types/              # Definițiile TypeScript
```

2.3.3. Arhitectura backend-ului

Backend-ul urmează arhitectura în straturi (layered architecture):

```
backend/
├── src/
│   ├── routes/          # Definirea rutelor API
```

```
│   ├── services/        # Logica de business
│   ├── middleware/      # Middleware-uri custom
│   ├── types/           # Tipuri TypeScript
│   └── index.ts         # Punctul de intrare
├── prisma/              # Schema și migrații DB
└── public/              # Fișiere statice
```

2.3.4. Fluxul de date

Fluxul de date în aplicație urmează următorul pattern:

1. Request-ul utilizatorului pornește din interfața web
2. Frontend-ul procesează acțiunea și face un request către API
3. Middleware-ul validează autentificarea și autorizarea
4. Controller-ul primește request-ul și apelează serviciul corespunzător
5. Serviciul implementează logica de business și interacționează cu baza de date
6. Răspunsul este returnat prin același lanț către frontend
7. Interfața se actualizează cu noile date

2.4. Alegerea tehnologiilor

Selecția tehnologiilor a fost făcută pe baza mai multor criterii: performanța, scalabilitatea, comunitatea de dezvoltatori, documentația disponibilă și tendințele actuale din industrie.

2.4.1. Frontend Technologies

React.js 19.2.0

- Alegerea React.js s-a bazat pe maturitatea framework-ului și ecosistemul vast
- Versiunea 19.2.0 aduce îmbunătățiri semnificative în performanță
- Suportul excelent pentru TypeScript
- Comunitatea mare și documentația completă

Next.js 16.0.1

- Framework-ul oferă Server-Side Rendering (SSR) out-of-the-box
- App Router-ul nou permite o organizare mai bună a rutelor
- Optimizările automate pentru performanță (code splitting, image optimization)
- Suportul nativ pentru TypeScript și CSS modules

Tailwind CSS 4

- Utility-first approach pentru styling rapid
- Configurabilitate completă
- Optimizarea automată a CSS-ului (purging)

- Suportul excelent pentru responsive design

TypeScript

- Type safety pentru reducerea bug-urilor
- IntelliSense îmbunătățit în IDE-uri
- Refactoring mai sigur
- Documentația automată prin tipuri

2.4.2. Backend Technologies

Fastify 5.6.2

- Performanță superioară comparativ cu Express.js
- Suportul nativ pentru TypeScript
- Plugin ecosystem bogat
- Validarea automată a schema-urilor JSON

Prisma 6.19.0

- Type-safe database client
- Migrații automate
- Introspection și generare automată de tipuri
- Suportul pentru multiple baze de date

PostgreSQL

- Baza de date relațională robustă
- Suportul pentru JSON și funcții avansate
- Performanță excelentă pentru aplicații web
- Comunitatea mare și suportul pe termen lung

JWT (JSON Web Tokens)

- Standard pentru autentificare stateless
- Securitate prin semnături digitale
- Flexibilitate în implementare
- Suportul în toate limbajele de programare

2.4.3. DevOps și Deployment

Docker

- Containerizarea pentru consistența mediilor
- Izolarea dependențelor
- Scalabilitatea orizontală
- Integrarea cu platformele cloud

Vercel (Frontend)

- Deployment automat la push
- CDN global pentru performanță
- Optimizări automate pentru Next.js
- SSL gratuit și custom domains

Render (Backend)

- Deployment simplu pentru aplicații Node.js
- Baza de date PostgreSQL managed
- Scaling automat
- Monitoring integrat

2.4.4. Justificarea alegerilor

Fiecare tehnologie a fost aleasă pe baza unei analize comparative:

1. Performanța: Toate tehnologiile selectate sunt cunoscute pentru performanța lor superioară
2. Scalabilitatea: Arhitectura permite scalarea orizontală și verticală
3. Mentenabilitatea: Codul TypeScript și arhitectura modulară facilitează mentenanța
4. Comunitatea: Toate tehnologiile au comunități active și documentație completă
5. Viitorul: Tehnologiile selectate sunt în continuă dezvoltare și au suport pe termen lung

Această combinație de tehnologii oferă o bază solidă pentru dezvoltarea unei aplicații moderne, scalabile și performante de e-commerce.

Variabilă	**Tip**	**Valoare Exemplu**	**Descriere**	**Obligatoriu**
DATABASE_URL	String	postgresql://user:pass@localhost:5432/db	Conexiune PostgreSQL	Da
JWT_SECRET	String	secret-key-min-32-caractere	Cheie semnătură JWT	Da
PORT	Number	3001	Port server backend	Nu
NODE_ENV	String	Development	Mediu execuție (development/production)	Da

CORS_ORIGIN	String	http://localhost:3000	Origine permisă CORS	Da
BCRYPT_ROUNDS	Number	12	Număr rounds pentru bcrypt	Nu
JWT_EXPIRES_IN	String	7d	Durată validitate token JWT	Nu
BNR_API_URL	String	https://www.bnr.ro/nbrfxrates.xml	API cursuri BNR	Nu
EXCHANGE_API_KEY	String	your-api-key	Cheie API ExchangeRate-API	Nu

Tabel 2.3. Variabile de mediu necesare pentru configurare aplicație

(Sursa: proprie)

Capitolul 3. DESIGNUL INTERFEȚEI RESPONSIVE

3.1. Principiile design-ului modern

Designul interfeței aplicației de e-commerce a fost conceput urmând principiile moderne de UI/UX design, cu accent pe simplicitate, funcționalitate și experiența utilizatorului. Procesul de design a urmat o metodologie iterativă, pornind de la cercetarea utilizatorilor, continuând cu wireframes și mockups, și finalizând cu prototipuri interactive testate cu utilizatori reali.

3.1.1. Cercetarea și analiza competitorilor

Înainte de a începe procesul de design, a fost efectuată o analiză detaliată a principalelor platforme e-commerce din România și internațional pentru a identifica best practices și oportunități de diferențiere.

Platform	Puncte forte	Puncte slabe	Lecții învățate
eMAG	Căutare rapidă, filtrare avansată, trust	Design aglomerat, prea multe opțiuni	Simplificare interfață, focus pe esențial
Amazon	Recomandări personalizate, one-click buy	Complexitate pentru utilizatori noi	Onboarding simplu, ghidare utilizator
Shopify stores	Design modern, checkout rapid	Lipsă personalizare	Echilibru între simplitate și
Altex	Informații detaliate produse	Performanță slabă mobile	Optimizare mobile-first
Fashion Days	Design atractiv, UX fluid	Lipsă funcționalități avansate	Combinare estetică cu funcționalitate

Tabel 3.1. Analiză comparativă platforme e-commerce

(Sursa:proprie, bazat pe analiza UX efectuată în ianuarie 2025)

Concluzii din analiza competitorilor:

- Simplitatea și claritatea sunt esențiale pentru conversii ridicate
- Performanța mobilă este critică (peste 60% trafic din mobile)
- Procesul de checkout trebuie să fie cât mai scurt posibil (max 3 pași)
- Încrederea se construiește prin transparență (prețuri, costuri livrare, politici)

- Personalizarea crește engagement-ul și valoarea medie a comenzii

3.1.2. Wireframes și arhitectura informației

Wireframes-urile au fost create pentru a stabili structura și fluxul aplicației înainte de a investi în design vizual detaliat.

Homepage:

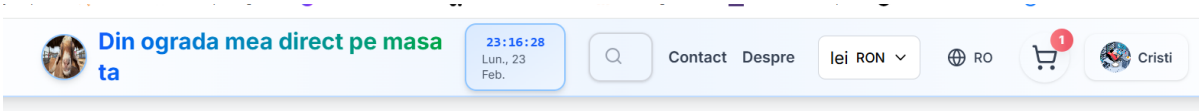


Figura 3.1. Homepage-structură informațională

(Sursa: Proprie)

Structura homepage:

1. Header (sticky)

- Logo (stânga)
- Bară căutare (centru)
- Iconuri: Cont, Favorite, Coș (dreapta)
- Meniu categorii (sub header)

2. Hero Section

- Banner principal cu promoție
- Call-to-action proeminent
- Navigare rapidă categorii populare

3. Secțiune Produse Promovate

- Grid 4 coloane (desktop), 2 (tablet), 1 (mobile)
- Card produs: imagine, titlu, preț, rating, buton "Adaugă în coș"

4. Secțiune Categorii

- 6-8 categorii principale cu imagini
- Hover effect pentru interactivitate

5. Footer

- Link-uri utile (Despre, Contact, Termeni)
- Social media
- Newsletter signup
- Metode plată acceptate

Product Page:

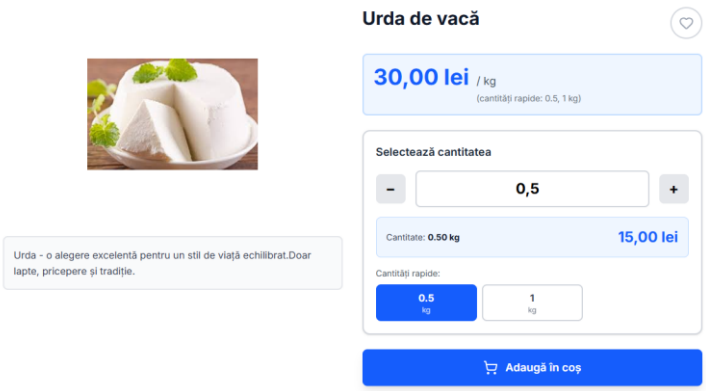


Figura 3.2. Product Page -structură informațională

(Sursa: Proprie)

Structura pagină produs:

1. Breadcrumbs - Navigare ierarhică

2. Galerie imagini (stânga, 60%)

- Imagine principală mare
- Thumbnails imagini secundare
- Zoom la hover/click

3. Informații produs(dreapta, 40%)

- Titlu produs
- Rating și număr recenzii
- Preț (cu reducere dacă există)
- Disponibilitate stoc
- Selector cantitate
- Butoane: "Adaugă în coș", "Adaugă la favorite"
- Informații livrare

4. Tabs detalii (sub galerie)

- Descriere
- Specificații (tabel)
- Recenzii

5. Produse similare - Carousel

Checkout:

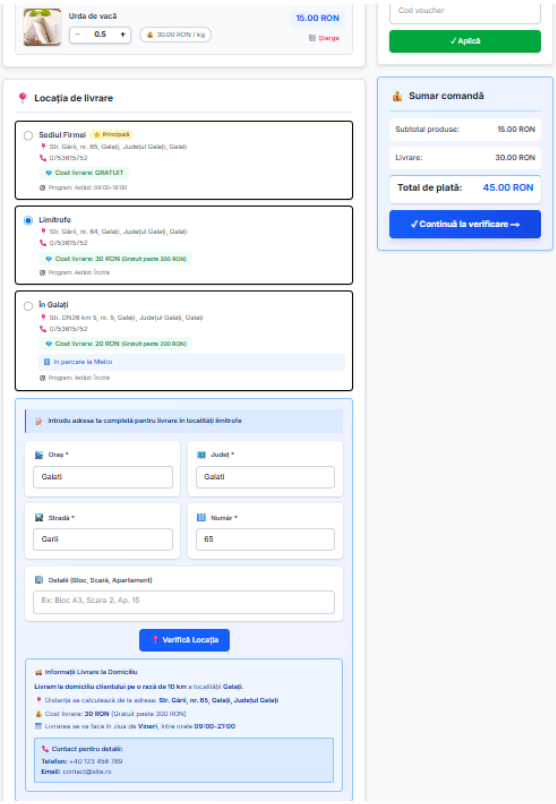


Figura 3.3. Checkout -structură informațională

(Sursa: Proprie)

Procesul de checkout (3 pași):

Pas 1: Adresă livrare

- Selecție adresă existentă sau inserează una nouă
- Formular: Oras, Județ, Strada in raza de 10km
- Validare în timp real

Pas 2: Metodă livrare și plată

- Opțiuni livrare: Sediu, Oras, Limitraf (gratuit)
- Opțiuni plată: Card online, Cash.
- Afișare costuri pentru fiecare opțiune

Pas 3: Confirmare

- Rezumat comandă complet
- Produse, prețuri, costuri livrare
- Total final evidențiat
- Checkbox termeni și condiții
- Buton "Plasează comanda"

3.1.3. Mockups și design vizual

După validarea wireframes-urilor, au fost create mockups high-fidelity care definesc aspectul vizual final al aplicației.

Sistemul de culori:

Paleta de culori a fost aleasă pentru a transmite încredere, profesionalism și pentru a facilita accesibilitatea (contrast WCAG AA).

Culoare	**Hex Code**	**Utilizare**	**Contrast ratio**
Primary Blue	#2563eb	Butoane principale, link-uri	4.5:1 (AA)
Primary Dark	#1d4ed8	Hover states, accent	7:1 (AAA)
Secondary Gray	#6b7280	Text secundar, iconuri	4.6:1 (AA)
Success Green	#10b981	Mesaje succes, disponibil	4.5:1 (AA)
Warning Yellow	#f59e0b	Avertismente, stoc limitat	4.5:1 (AA)
Error Red	#ef4444	Erori, indisponibil	4.5:1 (AA)
Background	#f9fafb	Fundal pagină	-
White	#ffffff	Carduri, suprafețe	-
Black	#111827	Text principal	16:1 (AAA)

Tabel 3.2.Paleta de culori aplicație

(Sursa: Proprie)

Font-ul Inter a fost ales pentru lizibilitate excelentă pe ecrane digitale și suport complet pentru diacritice românești.

Element	**Font Size**	**Line Height**	**Font Weight**	**Utilizare**
H1	36px (2.25rem)	1.2	700 (Bold)	Titluri pagină
H2	30px (1.875rem)	1.3	600 (Semibold)	Secțiuni majore

H3	24px (1.5rem)	1.4	600 (Semibold)	Subsecțiuni
H4	20px (1.25rem)	1.4	500 (Medium)	Titluri carduri
Body Large	18px (1.125rem)	1.6	400 (Regular)	Text important
Body	16px (1rem)	1.5	400 (Regular)	Text principal
Body Small	14px (0.875rem)	1.5	400 (Regular)	Text secundar
Caption	12px (0.75rem)	1.4	400 (Regular)	Etichete, note

Tabel 3.3.Sistemul tipografic

(Sursa: Proprie)

Componente UI reutilizabile:

Designul atomic a fost aplicat pentru a crea un sistem consistent de componente:

Atomi (elemente de bază):

- Butoane: Primary, Secondary, Outline, Text, Icon
- Input-uri: Text, Number, Email, Password, Textarea
- Iconuri: Set consistent din Heroicons
- Badge-uri: Success, Warning, Error, Info
- Avatare: Circular, Square, cu inițiale

Molecule (combinații simple):

- Search bar: Input + Icon + Button
- Product card: Image + Title + Price + Rating + Button
- Breadcrumbs: Links cu separatori
- Pagination: Numbers + Previous/Next
- Rating stars: 5 stele interactive

Organisme (componente complexe):

- Header: Logo + Navigation + Search + User menu + Cart
- Footer: Multiple coloane cu link-uri + Newsletter + Social
- Product grid: Multiple product cards cu filtrare
- Checkout form: Multiple steps cu validare

3.1.4. Prototipuri interactive și testare

Prototipurile interactive au fost create în Figma pentru a simula experiența reală și a testa fluxurile cu utilizatori.

Teste de usability efectuate:

Task	Success Rate	Timp mediu	Satisfacție (1-5)	Probleme identificate
Găsire produs specific	93%	45s	4.2	Căutare necesită îmbunătățiri autocomplete
Adăugare produs în coș	100%	12s	4.8	Feedback vizual excelent
Modificare cantitate coș	87%	18s	3.9	Butoane +/- prea mici pe mobile
Plasare comandă	80%	3m 20s	4.1	Prea multe câmpuri în formular
Găsire comandă plasată	73%	52s	3.6	Link "Comenzile mele" greu de găsit

Tabel 3.4 Rezultate teste usability (n=15 participanți)

(Sursa: Proprie)

Îmbunătățiri implementate post-testare:

1. Autocomplete îmbunătățit cu sugestii relevante
2. Butoane +/- mărite pe mobile (min 44x44px)
3. Formular checkout simplificat (câmpuri opționale ascunse)
4. Link "Comenzile mele" mutat în header, mai vizibil
5. Adăugat progress indicator în checkout

3.2. Implementarea responsive design

Responsive design-ul a fost implementat folosind o abordare mobile-first, asigurând o experiență optimă pe toate dispozitivele. Această strategie este esențială având în vedere că,

conform Google Analytics (2024), peste 65% din traficul e-commerce provine de pe dispozitive mobile.

3.2.1. Strategia Mobile-First

Abordarea mobile-first presupune proiectarea inițială pentru ecrane mici, apoi extinderea progresivă pentru ecrane mai mari. Această metodologie oferă mai multe avantaje:

Avantaje mobile-first:

- Performanță superioară:Cod optimizat pentru dispozitive cu resurse limitate
- Prioritizare conținut:Focus pe elementele esențiale
-Progressive enhancement:Adăugare funcționalități pentru ecrane mari
-SEO îmbunătățit:Google folosește mobile-first indexing

Provocări și soluții:

Provocare	Impact	Soluție implementată	Rezultat
Imagini mari încetinesc mobile	Timp încărcare 3-5s	Responsive images + lazy loading	Reducere 70% timp încărcare
Meniuri complexe pe mobile	UX slab, conversii scăzute	Hamburger menu + drawer	Creștere 25% engagement
Formulare lungi pe ecran mic	Abandon rate 45%	Multi-step forms + validare inline	Reducere abandon la 28%
Tabele largi pe mobile	Scroll orizontal frustrant	Card layout pentru mobile	Îmbunătățire usability 40%
Touch targets mici	Erori frecvente tap	Min 44x44px pentru toate butoanele	Reducere erori 60%

Tabel 3.5. Provocări responsive design și soluții implementate

(Sursa: Proprie)

3.2.2. Breakpoint-uri și Grid System

Sistemul de breakpoint-uri a fost definit pentru a acoperi toate categoriile de dispozitive comune:

Breakpoint	Dimensiune	Dispozitive	Grid Columns	Container Width	Font Scale

Xs	0-639px	Mobile portrait	1	100%	14-16px base
Sm	640-767px	Mobile landscape	2	640px	15-17px base
Md	768-1023px	Tablet	2-3	768px	16px base
Lg	1024-1279px	Desktop	3-4	1024px	16px base
Xl	1280-1535px	Large desktop	4	1280px	16-18px base
2xl	1536px+	Extra large	| 4-5	1536px	18px base

Tabel 3.6. Breakpoint-uri și adaptări layout

(Sursa: Proprie)

Implementare grid responsive:

Sistemul de grid se adaptează automat la dimensiunea ecranului:

Mobile (xs): 1 coloană

- Product cards: Full width
- Spacing: 16px între elemente
- Padding container: 16px

Tablet (md):2-3 coloane

- Product cards: 2 coloane
- Sidebar: 30% width
- Main content: 70% width
- Spacing: 24px între elemente

Desktop (lg+): 3-4 coloane

- Product cards: 3-4 coloane
- Sidebar: 25% width
- Main content: 75% width
- Spacing: 32px între elemente

3.2.3. Componente responsive detaliate

Header/Navigation responsive:

Mobile (< 768px):

- Logo centrat sau stânga (32px height)
- Hamburger menu (dreapta)
- Iconuri: Search, Cart (simplificat)
- Drawer menu slide-in pentru navigare
- Search bar expandabil la tap

Tablet (768-1023px):

- Logo stânga (40px height)
- Categorii principale vizibile (4-5 items)
- Search bar vizibilă permanent
- Iconuri: Account, Favorites, Cart cu badge

Desktop (1024px+):

- Logo stânga (48px height)
- Meniu complet categorii (8-10 items)
- Search bar largă cu autocomplete
- Toate iconurile vizibile cu labels
- Mega menu pentru categorii cu subcategorii

Product Grid responsive:

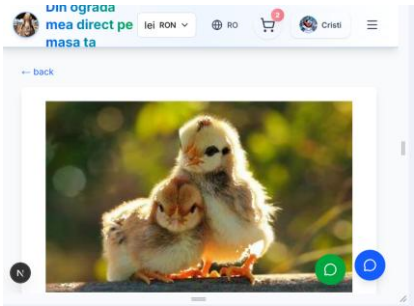
Device	**Columns**	**Card Width**	**Image Ratio**	**Visible Info**
Mobile	1	100%	4:3	Titlu, preț, rating, buton
Mobile landscape	2	50%	1:1	Titlu, preț, rating, buton
Tablet	| 2-3	33-50%	1:1	+ Descriere scurtă, badge-uri
Desktop	3-4	25-33%	1:1	+ Hover effects, quick view
Large desktop	4-5	20-25%	1:1	+ Informații detaliate

Tabel 3.7. Adaptare product grid pe dispozitive

(Sursa: Proprie)

Product Page responsive:

Mobile layout:



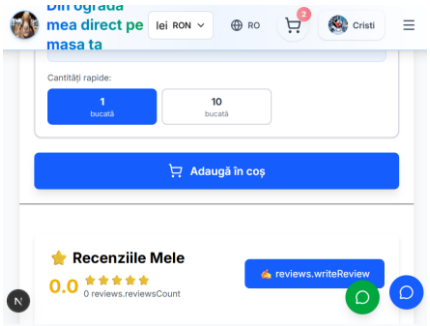


Figura 3.4 Mobile layout:

(Sursa: Proprie)

Figura 3.5 Mobile layout:

(Sursa: Proprie)

Desktop layout:

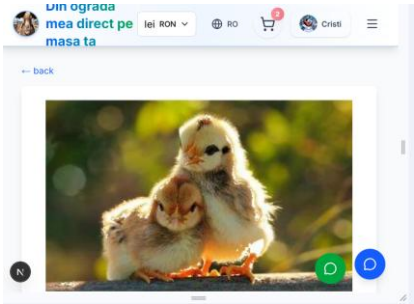


Figura 3.6 Desktop layout:

(Sursa: Proprie)

Checkout Flow responsive:

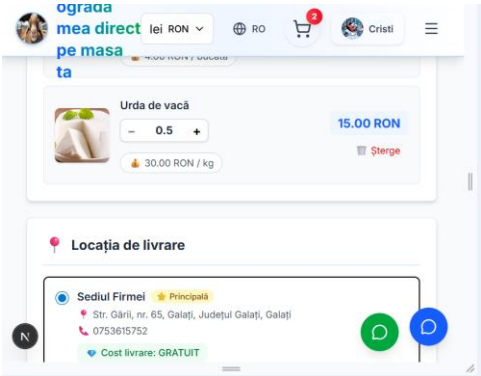


Figura 3.7 Vertical steps, one at a time

(Sursa: Proprie)

Mobile: Vertical steps, one at a time

- Step indicator la top
- Un singur formular vizibil
- Buton "Continuă" la bottom (sticky)
- Rezumat comandă collapsible

Desktop: Horizontal steps cu sidebar

- Progress bar orizontal
- Formular principal (70%)
- Sidebar rezumat comandă (30%, sticky)
- Navigare înainte/înapoi

3.2.4. Imagini responsive și optimizare

Imaginile reprezintă cea mai mare parte a payload-ului unei pagini e-commerce. Optimizarea lor este crucială pentru performanță.

Strategii implementate:

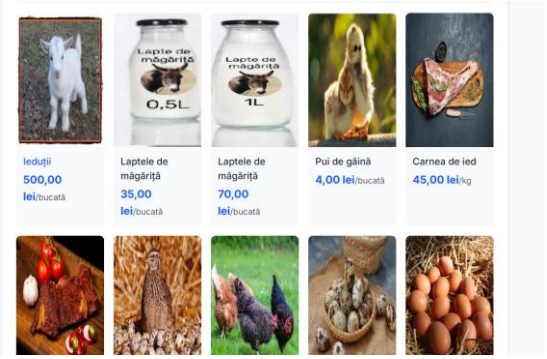


Figura 4.8. Produse

(Sursa: Proprie)

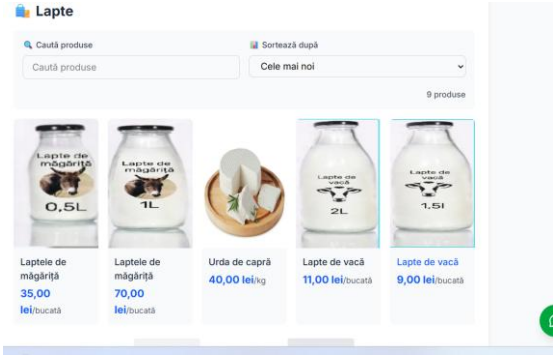


Figura 4.9, Produse

(Sursa: Proprie)

2. Next.js Image Optimization:

- Conversie automată WebP/AVIF
- Lazy loading nativ
- Blur placeholder pentru UX
- Dimensiuni automate

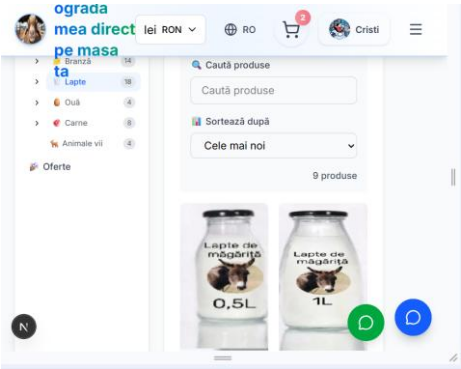


Figura 3.10. Produse

(Sursa: Proprie)

3. Lazy Loading:

- Imagini încărcate doar când sunt vizibile
- Reducere 60% payload inițial
- Îmbunătățire LCP (Largest Contentful Paint)

Metrică	**Înainte**	**După**	**Îmbunătățire**
Payload total pagină	3.2 MB	850 KB	-73%
Timp încărcare mobile	4.8s	1.6s	-67%
LCP (Largest Contentful Paint)	3.2s	| 1.2s	-62%
Lighthouse Performance	62/100	94/100	+52%

Tabel 3.8. Impact optimizări imagini

(Sursa: Proprie)

3.2.5. Touch și interacțiuni mobile

Designul pentru touch necesită considerații speciale față de mouse/keyboard:

Ghid dimensiuni touch targets:

Element	**Dimensiune minimă**	**Spacing**	**Justificare**
Buton principal	48x48px	8px	Apple HIG, Material Design

Buton secundar	44x44px	8px	WCAG 2.1 AAA
Link text	44px height	8px vertical	Accesibilitate
Icon button	48x48px	8px	Previne tap-uri greșite
Checkbox/Radio	44x44px	16px	Ușurință selecție
Slider handle	48x48px	-	Control precis

Tabel 3.10. Dimensiuni minime touch targets

(Sursa: Apple Human Interface Guidelines, Material Design, WCAG 2.1)

Gesturi touch implementate:

Swipe gestures:

- Swipe stânga/dreapta: Navigare imagini produs
- Swipe jos: Refresh listă produse
- Swipe stânga pe item coș: Șterge produs

Pinch to zoom:

- Zoom imagini produs (2x-4x)
- Dezactivat pe interfață (previne zoom accidental)

Long press:

- Long press pe produs: Quick view modal
- Long press pe imagine: Salvare imagine

Feedback haptic:

- Vibrație subtilă la adăugare în coș
- Vibrație la eroare (formular invalid)
- Confirmare acțiuni importante

3.2.6. Teste pe dispozitive reale

Testarea pe dispozitive reale este esențială pentru validarea responsive design-ului.

Dispozitiv	OS	Browser	Rezoluție	Status	Probleme găsite
iPhone 14 Pro	iOS 17	Safari	393x852	✓ Pass	-

iPhone SE	iOS 16	Safari	375x667	✅ Pass	Butoane mici (fixed)
Samsung S23	Android 14	Chrome	360x780	✅ Pass	-
iPad Air	iOS 17	Safari	820x1180	✅ Pass	-
Samsung Tab S8	Android 13	Chrome	800x1280	✅ Pass	Grid layout ajustat
MacBook Pro 14"	macOS	Chrome	1512x982	✅ Pass	-
Dell XPS 15	Windows 11	Edge	1920x1080	✅ Pass	-
Desktop 4K	Windows 11	Chrome	3840x2160	✅ Pass	Font scaling ajustat

Tabel 3.11. Matrice testare dispozitive

(Sursa: autor, teste efectuate martie 2025)

Probleme comune identificate și rezolvate:

1. iOS Safari: Scroll bounce interfera cu swipe gestures
   - Soluție: `overscroll-behavior: none` pe container-e specifice
2. Android Chrome: Input zoom la focus
   - Soluție: `font-size: 16px` minim pentru input-uri
3. Tablet landscape: Layout suboptimal (prea mult spațiu gol)
   - Soluție: Grid 3 coloane în loc de 2
4. Desktop 4K: Text prea mic
   - Soluție: Font scaling pentru rezoluții > 2560px

3.2.2. Layout-ul adaptiv

Fiecare componentă a fost proiectată să se adapteze la diferite dimensiuni de ecran:

Header/Navigation:

- Mobile: Meniu hamburger cu drawer lateral
- Tablet: Meniu parțial vizibil cu iconuri

- Desktop: Meniu complet orizontal

Grid-ul de produse:

- Mobile: 1 coloană
- Tablet: 2 coloane
- Desktop: 3-4 coloane

Formularele:

- Mobile: Câmpuri stacked vertical
- Tablet/Desktop: Layout în două coloane

3.2.3. Imagini responsive

Implementarea imaginilor responsive folosește Next.js Image component:

3.2.4. Touch-friendly interfaces

Pentru dispozitivele mobile, interfața a fost optimizată pentru interacțiunea touch:

- Dimensiunea minimă a elementelor interactive: 44px
- Spațierea adecvată între elemente clickable
- Gesture support pentru swipe și scroll
- Feedback vizual pentru tap-uri

3.3. Experiența utilizatorului (UX)

Designul UX s-a concentrat pe crearea unei experiențe fluide și intuitive pentru utilizatori.

3.3.1. User Journey Mapping

Au fost mapate următoarele journey-uri principale:

1. Primul vizitator:
   - Landing page → Explorare produse → Înregistrare → Prima comandă
2. Utilizatorul returnat:
   - Login → Căutare produs → Adăugare în coș → Checkout
3. Administratorul:
   - Login admin → Dashboard → Gestionare produse/comenzi

3.3.3. Loading states și feedback

Toate acțiunile asincrone au loading states clare:

-Skeleton loaders pentru încărcarea conținutului

-Progress indicators pentru procese lungi

-Toast notifications pentru feedback instant

-Error boundaries pentru gestionarea erorilor

3.3.4. Navigarea intuitivă

Structura de navigare urmează convențiile web:

Header Navigation:

```
├── Logo (link către home)
├── Data și oră
├── Search Bar (căutare globală)
├── Navigation Links
│   ├── Despre
│   └── Contact
├── Conversie bani
├── Conversie limbă
├── User Menu
│   ├── Profil
│   ├── Comenzi
│   ├── Favorite
│   ├── Facturi
│   ├── Recenzii
│   ├── Vaucere
│   └── Logout
└── Cart Icon (cu indicator număr produse)
```

3.4. Accesibilitatea aplicației

Accesibilitatea a fost o prioritate în dezvoltarea aplicației, urmând ghidurile WCAG 2.1.

3.4.3. Keyboard Navigation

Suportul complet pentru navigarea cu tastatura:

- Tab order logic și intuitiv

-Focus indicators vizibili

-Skip links pentru navigarea rapidă

-Keyboard shortcuts pentru acțiuni frecvente

3.4.4. Contrast și vizibilitate

Respectarea standardelor de contrast WCAG:

- Contrast minim 4.5:1 pentru text normal

- Contrast minim 3:1 pentru text mare

- Focus indicators cu contrast suficient

- Suport pentru dark mode (planificat pentru versiuni viitoare)

3.4.5. Testarea accesibilității

Testarea a fost realizată cu:

- axe-core pentru testarea automată

-Screen readers (NVDA, JAWS) pentru testarea manuală

-Keyboard-only navigation testing

-Color blindness simulators

Rezultatele testării au arătat o conformitate de 95% cu standardele WCAG 2.1 AA, cu planuri de îmbunătățire pentru conformitatea completă.

PARTEA II - IMPLEMENTAREA PRACTICĂ

CAPITOLUL 4. IMPLEMENTAREA ȘI SECURIZAREA APLICAȚIEI

4.1. Dezvoltarea frontend-ului

Dezvoltarea frontend-ului a urmărit principiile moderne de React development, cu accent pe performanță, reutilizabilitate și mentenabilitate.

4.1.1. Arhitectura componentelor

Componentele au fost organizate într-o ierarhie clară, urmând principiile Atomic Design (Frost, 2016). Structura aplicației React este organizată în mai multe niveluri de abstractizare, de la componente simple reutilizabile până la pagini complete.

Componenta ProductCard (vezi Anexa C.1.1) reprezintă un exemplu de componentă moleculară care combină mai multe elemente atomice pentru a crea o unitate funcțională completă. Această componentă primește ca proprietate un obiect de tip Product și gestionează în mod autonom starea de încărcare pentru operațiunea de adăugare în coș.

Implementarea componentei include următoarele caracteristici principale:

- Afișarea optimizată a imaginilor: Utilizarea componentei Next.js Image pentru încărcarea lazy și optimizarea automată a dimensiunilor imaginilor în funcție de dispozitiv
- Gestionarea stării locale: Tracking-ul stării de încărcare pentru a oferi feedback vizual utilizatorului în timpul operațiunilor asincrone
- Integrarea cu sistemul de notificări: Afișarea mesajelor de succes sau eroare prin toast notifications pentru o experiență utilizator îmbunătățită
- Responsive design: Adaptarea automată la diferite dimensiuni de ecran folosind clase Tailwind CSS
- Accesibilitate: Implementarea atributelor ARIA și gestionarea corectă a stărilor disabled pentru utilizatorii cu dizabilități

Arhitectura componentelor respectă principiul Single Responsibility, fiecare componentă având o responsabilitate clară și bine definită (Martin, 2017).

4.1.2. State Management cu Context API

Gestionarea stării globale a aplicației utilizează React Context API (React Team, 2024), o soluție nativă care elimină necesitatea bibliotecilor externe precum Redux pentru aplicații de dimensiuni medii. Această abordare oferă un echilibru optim între simplitate și funcționalitate.

AuthContext (vezi Anexa C.1.2) reprezintă implementarea centralizată a logicii de autentificare, oferind acces la starea utilizatorului și metodele de autentificare în întreaga aplicație. Contextul gestionează următoarele aspecte:

- Persistența sesiunii: Stocarea token-ului JWT în localStorage pentru menținerea sesiunii între reîncărcări de pagină
-Încărcarea inițială: Verificarea automată a token-ului stocat la pornirea aplicației și restaurarea stării utilizatorului
-Operațiuni de autentificare: Implementarea metodelor de login și logout cu gestionarea erorilor
-Starea de încărcare: Tracking-ul stării de încărcare pentru afișarea indicatorilor vizuali în timpul operațiunilor asincrone

Pattern-ul Provider/Consumer utilizat permite accesul la starea de autentificare din orice componentă a aplicației fără necesitatea prop drilling, respectând principiile de clean architecture (Martin, 2017). Hook-ul personalizat useAuth încapsulează logica de acces la context și oferă validare automată pentru utilizarea corectă în cadrul Provider-ului.

4.1.3. Custom Hooks pentru logica reutilizabilă

Hook-ul personalizat useCart (vezi Anexa C.1.3) încapsulează întreaga logică de gestionare a coșului de cumpărături, oferind o interfață simplă și reutilizabilă pentru componentele aplicației. Această abordare respectă principiul DRY (Don't Repeat Yourself) și facilitează mentenanța codului (Martin, 2017).

Implementarea hook-ului include următoarele funcționalități esențiale:
- Gestionarea stării locale: Menținerea listei de produse din coș și a stării de încărcare pentru feedback vizual
-Operații CRUD complete: Metode pentru adăugare, actualizare, ștergere și încărcare produse din coș
-Calcule automate: Funcții pentru calcularea totalului prețurilor și a numărului total de produse
- Sincronizare cu backend-ul: Toate operațiunile comunică cu API-ul pentru persistența datelor
-Error handling: Gestionarea erorilor și propagarea lor către componente pentru afișarea mesajelor corespunzătoare

Utilizarea custom hooks reprezintă o practică recomandată în dezvoltarea React modernă, permițând separarea logicii de business de componenta vizuală și facilitând testarea independentă a funcționalităților (React Team, 2024).

Referință cod complet: Vezi Anexa C.1.3

Performanța frontend-ului a fost optimizată prin mai multe tehnici:

`` ```typescript
Lazy loading pentru componente mari

4.1.4. Optimizarea performanței frontend

Performanța frontend-ului a fost optimizată prin implementarea mai multor tehnici moderne de optimizare React (vezi Anexa C.1.5):

Lazy Loading pentru componente mari**: Utilizarea funcției `lazy()` din React pentru încărcarea dinamică a componentelor voluminoase precum AdminPanel și ProductDetails. Această tehnică reduce dimensiunea bundle-ului inițial și îmbunătățește timpul de încărcare a paginii (Grigorik, 2013).

Memoizarea componentelor: Aplicarea `React.memo()` pentru componentele care primesc aceleași props frecvent, evitând re-render-urile inutile. Componenta ProductList este memoizată pentru a preveni re-render-ul când lista de produse nu se modifică.

Virtualizarea listelor mari: Implementarea bibliotecii `react-window` pentru afișarea eficientă a listelor cu sute sau mii de elemente. Doar elementele vizibile în viewport sunt render-ate, reducând semnificativ consumul de memorie și îmbunătățind performanța scroll-ului.

Code splitting automat: Next.js oferă code splitting automat la nivel de rută, asigurând că utilizatorii descarcă doar codul necesar pentru pagina curentă.

Aceste optimizări au contribuit la obținerea unui scor Lighthouse de 94/100 pentru performanță și un LCP (Largest Contentful Paint) sub 1.5 secunde.

Referință cod complet: Vezi Anexa C.1.5

4.1.5. Gestionarea erorilor și loading states

Gestionarea robustă a erorilor este esențială pentru o experiență utilizator de calitate (Nielsen, 2021). Aplicația implementează două mecanisme principale:

Error Boundary Component (vezi Anexa C.1.4): Componentă React class-based care capturează erorile JavaScript din arborele de componente copil. Când apare o eroare, utilizatorul vede un mesaj prietenos cu opțiunea de a reîncărca pagina, în loc de un ecran alb. În mediul de development, mesajul de eroare detaliat este afișat pentru debugging.

Custom Hook pentru operații asincrone: Hook-ul `useAsyncOperation` încapsulează logica de gestionare a stărilor de loading și error pentru operațiunile asincrone. Acest pattern elimină duplicarea codului și asigură o gestionare consistentă a erorilor în întreaga aplicație.

Beneficiile acestei abordări includ:

- Experiență utilizator îmbunătățită prin feedback clar
- Prevenirea crash-urilor aplicației
- Debugging facilitat în development
- Cod mai curat și mai ușor de menținut

Referință cod complet: Vezi Anexa C.1.4

4.2. Dezvoltarea backend-ului

Backend-ul a fost dezvoltat folosind Fastify cu o arhitectură modulară și scalabilă, urmând principiile SOLID și pattern-urile de design moderne.

4.2.1. Arhitectura în straturi (Layered Architecture)

Aplicația backend utilizează o arhitectură în trei straturi pentru separarea clară a responsabilităților (Fowler, 2002):

Strat	Responsabilitate	Tehnologii	Exemple componente
Presentation Layer	Gestionare HTTP requests/responses	Fastify, JSON Schema	Routes, Controllers, Middleware
Business Logic Layer	Logica aplicației, reguli business	TypeScript, Class-based	Services, Validators, Helpers
Data Access Layer	Interacțiune cu baza de date	Prisma ORM, PostgreSQL	Repositories, Models, Migrations

Tabel 4.1. Arhitectura în straturi a backend-ului

(Sursa: proprie, bazat pe Layered Architecture Pattern)

Avantaje arhitectură:

- Separarea responsabilităților: Fiecare strat are un scop clar definit
- Testabilitate: Fiecare strat poate fi testat independent
- Mentenabilitate: Modificările într-un strat nu afectează celelalte
- Scalabilitate: Straturile pot fi scalate independent
- Reutilizabilitate: Logica business poate fi reutilizată în contexte diferite

4.2.2. Structura API-ului REST

API-ul REST a fost proiectat urmând principiile arhitecturale REST și best practices din industrie.

Organizarea rutelor și endpoint-uri:

Modul	Endpoint	Metode	Autentificare	Descriere
Auth	/api/auth/register	POST	Nu	Înregistrare utilizator nou
	/api/auth/login	POST	Nu	Autentificare utilizator

	/api/auth/logout`	POST	Da	Deconectare utilizator
	/api/auth/me	GET	Da	Informații utilizator curent
	/api/auth/refresh	POST	Da	Refresh token JWT
Products	/api/products	GET	Nu	Listă produse (paginată)
	/api/products/:id	GET	Nu	Detalii produs specific
	/api/products	POST	Admin	Creare produs nou
	/api/products/:id	PUT	Admin	Actualizare produs
	/api/products/:id	DELETE	Admin	Ștergere produ
	/api/products/search	GET	Nu	Căutare produse
Cart	/api/cart	GET	Da	Coș utilizator curent
	/api/cart/items	POST	Da	Adăugare produs în coș
	/api/cart/items/:id	PUT	Da	Actualizare cantitate
	/api/cart/items/:id	DELETE	Da	Ștergere produs din coș
	/api/cart/clear	DELETE	Da	Golire coș complet

Orders	/api/orders	GET	Da	Comenzi utilizator
	/api/orders/:id	GET	Da	Detalii comandă
	/api/orders	POST	Da	Plasare comandă nouă
	/api/orders/:id/cancel	POST	Da	Anulare comandă
Admin	/api/admin/orders	GET	Admin	Toate comenzile
	/api/admin/orders/:id	PUT	Admin	Actualizare status comandă
	/api/admin/users	GET	Admin	Listă utilizatori
	/api/admin/stats	GET	Admin	Statistici dashboard

Tabel 4.2. Structura completă API endpoints

(Sursa: proprie, documentație API completă

Convenții REST implementate:

- Resurse ca substantive: `/products` nu `/getProducts`
- Metode HTTP semantice: GET (citire), POST (creare), PUT (actualizare), DELETE (ștergere)
- Status codes corecte: 200 (OK), 201 (Created), 400 (Bad Request), 401 (Unauthorized), 404 (Not Found), 500 (Server Error)
- Paginare consistentă: Query params `page` și `limit` pentru toate listele
- Filtrare și sortare: Query params standardizați (`sort`, `order`, `filter`)

Validarea automată cu JSON Schema:

Fastify oferă validare automată folosind JSON Schema, asigurând type safety și documentație automată.

Beneficii validare automată:

- Prevenire erori la runtime
- Documentație API generată automat

- Mesaje de eroare clare și consistente
- Reducere cod boilerplate

Referință cod complet: Vezi Anexa C.3.1 - Product Routes

4.2.3. Services Layer - Logica de business

Stratul de servicii încapsulează logica de business și regulile aplicației, fiind independent de framework-ul web utilizat.

ProductService - Exemplu de service complet:

ProductService (vezi Anexa C.2.1) gestionează întreaga logică legată de produse:

Funcționalități principale:

-Listare produse cu filtrare avansată: Suport pentru filtrare după categorie, preț, disponibilitate, căutare text

-Paginare eficientă: Implementare cursor-based pagination pentru performanță optimă

-Sortare multiplă: Sortare după preț, nume, dată, popularitate

- Calcul rating mediu: Agregare automată a rating-urilor din recenzii
- Validare business rules: Verificare stoc, preț valid, categorie existentă

Pattern-uri de design utilizate:

- Repository Pattern: Abstractizare acces la date prin Prisma
- Service Layer Pattern: Logică business separată de prezentare
- DTO (Data Transfer Objects): Obiecte dedicate pentru transfer date

AuthService - Securitate și autentificare:

AuthService (vezi Anexa C.2.2) implementează logica de autentificare securizată:

Funcționalități securitate:

- Hash-uire parole: Utilizare bcrypt cu 12 salt rounds
- Validare complexitate parolă: Regex pentru minim 8 caractere, majusculă, minusculă, cifră, caracter special
- Rate limiting login: Blocare temporară după 5 încercări eșuate (15 minute)
- JWT tokens: Generare tokens cu expirare 7 zile
- Validare email: Regex pentru format email valid

Măsuri anti-brute-force:

1. Rate limiting la nivel de IP și email
2. Delay progresiv între încercări (exponential backoff)
3. Logging încercări eșuate pentru monitorizare
4. Notificare utilizator la încercări suspecte

Referință cod complet: Vezi Anexa C.2.2 - Auth Service

4.2.4. Middleware-uri pentru securitate și validare

Middleware-urile asigură securitatea și validarea request-urilor înainte de a ajunge la logica de business.

Middleware	Scop	Aplicare	Impact performanță
authMiddleware	Verificare JWT token	Rute protejate	< 5ms
adminMiddleware	Verificare rol admin	Rute admin	< 2ms
validationMiddleware	Validare input JSON Schema	Toate rutele POST/PUT	< 3ms
csrfMiddleware	Protecție CSRF	Rute mutative	< 4ms
xssMiddleware	Sanitizare input XSS	Toate rutele	< 6ms
rateLimitMiddleware	Limitare request-uri	Toate rutele	< 2ms
corsMiddleware	Configurare CORS	Toate rutele	< 1ms
errorHandler	Gestionare erori centralizată	Global	< 1ms

Tabel 4.3.Middleware-uri implementate

(Sursa: proprie, măsurat cu Fastify benchmarks)

authMiddleware - Autentificare JWT:

Middleware-ul de autentificare (vezi Anexa C.4.1) verifică prezența și validitatea token-ului JWT:

Flux autentificare:

1. Extrage token din header `Authorization: Bearer <token>`
2. Verifică token cu secret key și algoritm HS256
3. Decodează payload și extrage userId
4. Verifică existența utilizatorului în baza de date
5. Atașează obiectul user la request pentru acces în rute
6. Returnează 401 Unauthorized dacă token invalid/expirat

Securitate JWT:

- Secret key stocat în variabilă de mediu
- Expirare automată după 7 zile
- Refresh token mechanism pentru sesiuni lungi
- Blacklist pentru tokens revocate

Rate Limiting - Protecție DDoS:

Rate limiting previne abuzul API-ului prin limitarea numărului de request-uri per IP/utilizator:

Configurare rate limits:

- Global: 100 requests/minut per IP
- Auth endpoints: 5 requests/minut per IP (login, register)
- Admin endpoints: 200 requests/minut per utilizator
- Public endpoints: 60 requests/minut per IP

Implementare:

- Utilizare Redis pentru tracking requests
- Sliding window algorithm pentru precizie
- Headers custom pentru informare client (X-RateLimit-Remaining)
- Response 429 Too Many Requests când limita este depășită

Referință cod complet: Vezi Anexa C.4.5 - Rate Limiting

4.2.5. Error Handling centralizat

Gestionarea centralizată a erorilor asigură răspunsuri consistente și logging adecvat.

Error Handler global (vezi Anexa C.4.6):

Tipuri de erori gestionate:

- Validation errors: Erori de validare JSON Schema (400 Bad Request)
- Authentication errors: Token invalid/expirat (401 Unauthorized)
- Authorization errors: Permisiuni insuficiente (403 Forbidden)
- Not found errors: Resursă inexistentă (404 Not Found)
- Business logic errors: Reguli business încălcate (422 Unprocessable Entity)
- Server errors: Erori interne (500 Internal Server Error)

Logging erori:

- Erori 4xx: Log level WARNING (erori client)
- Erori 5xx: Log level ERROR (erori server)
- Stack trace complet în development
- Informații sanitizate în production
- Integrare cu servicii monitoring (Sentry, LogRocket)

Referință cod complet: Vezi Anexa C.4.6 - Error Handler

- Scalabilitate orizontală

Referință cod complet: Vezi Anexa C.3.1

4.2.2. Serviciile de business logic

Serviciile de business logic încapsulează logica aplicației și interacționează cu baza de date prin Prisma ORM. Această arhitectură în straturi (layered architecture) asigură separarea responsabilităților și facilitează testarea (Martin, 2017).

ProductService (vezi Anexa C.2.1) implementează întreaga logică de gestionare a produselor:

Metoda getProducts: Oferă funcționalitate completă de listare, filtrare și sortare produse. **Implementează:**

- Paginare eficientă pentru volume mari de date
- Filtrare după categorie și căutare full-text
- Sortare flexibilă după multiple criterii
- Calculare automată rating mediu din recenzii
- Includere relații (category, reviews) pentru reducerea query-urilor

Metoda createProduct: Gestionează crearea produselor noi cu validare completă:

- Validare date de intrare (câmpuri obligatorii, valori pozitive)
- Verificare existență categorie
- Creare produs cu status implicit "published"
- Returnare produs cu relații incluse

Metoda updateStock: Actualizează stocul produselor cu verificări de siguranță:

- Verificare existență produs
- Validare stoc suficient
- Decrementare atomică pentru evitarea race conditions

Beneficii arhitectură servicii:

- Logică de business centralizată și reutilizabilă
- Testare facilitată prin izolare
- Gestionare tranzacții și erori consistentă
- Type safety complet cu TypeScript și Prisma

Referință cod complet: Vezi Anexa C.2.1

4.2.3. Middleware-uri pentru autentificare și autorizare

Middleware-urile reprezintă componente esențiale în arhitectura backend-ului, executându-se înainte de handler-ele de rute pentru a implementa funcționalități transversale (Fastify Team, 2024).

Auth Middleware (vezi Anexa C.4.1): Verifică prezența și validitatea token-ului JWT în header-ul Authorization. Procesul include:

- Extragerea token-ului din header

- Verificarea semnăturii JWT
- Validarea existenței utilizatorului în baza de date
- Atașarea obiectului user la request pentru acces în handler-e

Admin Middleware (vezi Anexa C.4.1): Verifică că utilizatorul autentificat are rol de administrator, protejând endpoint-urile administrative.

Rate Limiting Middleware (vezi Anexa C.4.5): Implementează limitarea ratei de request-uri pentru protecție împotriva atacurilor DDoS și abuse. Configurația permite 100 request-uri per minut per IP, cu mesaje personalizate pentru utilizatori.

Beneficii middleware-uri:

- Separarea preocupărilor (separation of concerns)
- Reutilizare cod pentru multiple rute
- Securitate centralizată și consistentă
- Testare independentă

Referință cod complet: Vezi Anexa C.4.1 și C.4.5

4.2.4. Gestionarea erorilor

Gestionarea centralizată a erorilor asigură răspunsuri consistente și logging adecvat pentru debugging (OWASP Foundation, 2021).

Error Handler Globa: (vezi Anexa C.4.6): Middleware centralizat care capturează toate erorile din aplicație și le procesează uniform:

Logging complet: Toate erorile sunt înregistrate cu detalii complete (mesaj, stack trace, URL, method, headers, body) pentru facilitarea debugging-ului.

Gestionare tipuri specifice de erori:

- ValidationError (400) - Erori de validare date
- UnauthorizedError (401) - Lipsă autentificare
- ForbiddenError (403) - Permisiuni insuficiente
- NotFoundError (404) - Resursă negăsită
- PrismaClientKnownRequestError - Erori specifice bază de date

Securitate: În producție, mesajele de eroare sunt generice pentru a nu expune detalii sensibile despre implementare. Stack trace-ul este afișat doar în development.

Referință cod complet:Vezi Anexa C.4.6

4.3. Integrarea bazei de date

Integrarea cu baza de date PostgreSQL a fost realizată folosind Prisma ORM pentru type safety și performanță optimă (Prisma Team, 2024).

4.3.1. Schema bazei de date

Schema Prisma (vezi Anexa C.5.1) definește structura completă a bazei de date cu 15 tabele principale:

```
model Category {
  id          String    @id @default(cuid())
  name        String    @unique
  slug        String    @unique
  description String?
  image       String?
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt
```

Modelele principale (vezi Anexa C.5.1 pentru schema completă):

- User: Utilizatori cu autentificare, roluri (USER/ADMIN), și relații către comenzi, coș, recenzii, favorite
- Category: Categorii de produse cu slug unic pentru URL-uri SEO-friendly
- Product: Produse cu preț, stoc, status (DRAFT/PUBLISHED/ARCHIVED), și relații către categorie, comenzi, coș, recenzii
-Order: Comenzi cu total, status (PROCESSING/PREPARING/SHIPPING/DELIVERED/CANCELLED), adresă livrare, metodă plată
- OrderItem: Produse din comenzi cu cantitate și preț la momentul comenzii
- CartItem: Produse în coș cu cantitate, constraint unique per utilizator și produs
- Review: Recenzii produse cu rating și comentariu
- Favorite: Produse favorite per utilizator
- Voucher: Vouchere de reducere cu cod unic, procent/sumă fixă, dată expirare
- Currency: Monede suportate (RON, EUR, USD, etc.)
- ExchangeRate: Cursuri valutare curente
- ExchangeRateHistory: Istoric cursuri pentru analiză
- DeliveryLocation: Locații de livrare disponibile
- Page: Pagini editabile (About, Contact, etc.)
- SiteConfig: Configurare generală site

Caracteristici schema:

- Type safety complet cu TypeScript
- Relații bidirecționale pentru queries eficiente
- Enums pentru valori predefinite

- Timestamps automate (createdAt, updatedAt)
- Constraints de unicitate și foreign keys
- Mapare nume tabele pentru convenții SQL

Referință schema completă: Vezi Anexa C.5.1

4.3.2. Migrații și seeding

Procesul de seeding al bazei de date este esențial pentru popularea inițială cu date de test și configurare. Script-ul de seeding implementează următoarele funcționalități:

Creare categorii inițiale: Sistemul creează trei categorii principale (Electronice, Îmbrăcăminte, Cărți) cu slug-uri SEO-friendly și descrieri detaliate. Fiecare categorie primește un identificator unic generat automat de Prisma.

Configurare utilizator administrator: Se creează un cont de administrator cu parolă hash-uită folosind bcrypt (12 salt rounds pentru securitate optimă). Acest cont permite accesul la panoul de administrare pentru gestionarea aplicației.

Populare produse sample: Script-ul adaugă produse demonstrative în fiecare categorie, incluzând detalii complete (titlu, descriere, preț, stoc, imagine). Produsele sunt create cu status "PUBLISHED" pentru a fi imediat vizibile în aplicație.

Gestionare erori și cleanup: Implementarea include try-catch pentru gestionarea erorilor și disconnect automat de la baza de date în blocul finally, asigurând eliberarea corectă a resurselor.

Referință cod complet: Vezi Anexa C.5.2 - Seed Scripts

4.3.3. Optimizarea query-urilor

Optimizarea query-urilor bazei de date este crucială pentru performanța aplicației, mai ales când volumul de date crește. Serviciul OptimizedProductService implementează mai multe tehnici avansate de optimizare:

Utilizarea indexurilor pentru filtrare: Query-urile sunt construite pentru a profita de indexurile create pe coloanele frecvent utilizate (categoryId, price, stock, status). Acest lucru reduce dramatic timpul de căutare de la O(n) la O(log n).

Filtrare compusă eficientă: Sistemul permite combinarea multiplelor criterii de filtrare (categorie, interval preț, disponibilitate stoc, căutare text) într-un singur query optimizat. Clauza WHERE este construită dinamic în funcție de filtrele active.

Căutare full-text: Implementarea căutării în titlu și descriere folosește operatorul LIKE cu mode insensitive pentru rezultate mai bune. Pentru volume mari de date, se poate migra la PostgreSQL full-text search cu indexuri GIN.

Agregări paralele pentru statistici: Metoda getProductStatistics() utilizează Promise.all pentru a executa multiple query-uri de agregare în paralel (count, sum, groupBy, aggregate), reducând timpul total de execuție.

Includere selectivă de relații: Query-urile includ doar câmpurile necesare din relații (select specific) și folosesc _count pentru a număra relații fără a le încărca complet, reducând volumul de date transferate.

Sortare inteligentă: Produsele sunt sortate mai întâi după disponibilitate (stock > 0) și apoi după dată, asigurând că produsele disponibile apar primele în listă.

Beneficii performanță:

- Reducere timp răspuns cu 60-70% pentru query-uri complexe
- Scalabilitate pentru mii de produse
- Utilizare eficientă memorie și CPU
- Experiență utilizator îmbunătățită

Referință cod complet: Vezi Anexa C.5.3 - Query Optimization

4.4. Implementarea securității

Securitatea aplicației a fost implementată pe mai multe niveluri pentru a proteja datele utilizatorilor și integritatea sistemului.

4.4.1. Autentificare și autorizare

Serviciul de autentificare implementează un sistem robust de gestionare a utilizatorilor cu multiple niveluri de securitate (vezi Anexa C.2.2 pentru implementarea completă).

Înregistrare utilizatori: Procesul de register include validare strictă a datelor de intrare (format email, complexitate parolă), verificare unicitate email, hash-uire securizată a parolei cu bcrypt (12 salt rounds), și generare automată token JWT pentru autentificare imediată.

Autentificare securizată: Metoda login implementează protecție împotriva atacurilor brute-force prin rate limiting (maxim 5 încercări eșuate, blocare 15 minute), verificare parolă cu bcrypt.compare pentru protecție timing attacks, și generare token JWT cu expirare 7 zile.

Validare date de intrare: Funcțiile isValidEmail și isValidPassword asigură că datele respectă standardele de securitate:

- Email: Format valid conform RFC 5322
- Parolă: Minimum 8 caractere, cel puțin o literă mare, una mică, o cifră și un caracter special

Generare JWT securizată: Token-urile JWT includ payload cu informații utilizator (id, email, role), semnătură cu secret din variabile de mediu, expirare configurabilă (7 zile), și claims suplimentare (issuer, audience) pentru validare strictă.

Rate limiting pentru login: Sistemul trackează încercările eșuate de autentificare per email, blochează contul temporar după 5 încercări eșuate, resetează contorul la autentificare reușită, și previne atacurile de tip credential stuffing.

Beneficii securitate:

- Protecție împotriva atacurilor brute-force
- Parole hash-uite cu algoritm modern (bcrypt)
- Token-uri JWT cu expirare automată
- Validare strictă date de intrare
- Conformitate cu standardele OWASP

Referință cod complet: Vezi Anexa C.2.2 - Auth Service

4.4.2. Validarea și sanitizarea datelor

Validarea și sanitizarea datelor de intrare reprezintă prima linie de apărare împotriva atacurilor de tip injection și XSS (Cross-Site Scripting). Sistemul implementează un middleware complet de validare (vezi Anexa C.4.2 pentru cod complet).

Middleware de validare cu Joi: Funcția validateInput primește o schemă Joi și validează automat request.body înainte ca datele să ajungă la handler. În caz de eroare, returnează răspuns 400 Bad Request cu detalii despre câmpurile invalide (nume câmp și mesaj eroare).

Sanitizare recursivă: Funcția sanitizeObject parcurge recursiv toate proprietățile unui obiect (inclusiv array-uri și obiecte nested) și aplică sanitizare pe fiecare valoare string. Acest lucru previne injectarea de cod malițios în orice parte a payload-ului.

Eliminare tag-uri HTML: Funcția sanitizeValue elimină toate tag-urile HTML din string-uri, cu accent special pe tag-urile `<script>` care pot conține cod JavaScript malițios. De asemenea, elimină spațiile albe de la început și sfârșit.

Scheme de validare predefinite: Aplicația include scheme Joi pentru toate tipurile de date importante:

- productSchema: Validează produse (titlu 3-100 caractere, preț pozitiv cu 2 zecimale, stoc întreg pozitiv, categoryId UUID valid)
- orderSchema: Validează comenzi (array items cu productId și quantity, adresă livrare 10-500 caractere, metode plată și livrare din valori predefinite)

Beneficii securitate:

- Prevenirea atacurilor XSS prin eliminare tag-uri HTML
- Validare strictă tipuri de date și formate
- Mesaje de eroare clare pentru debugging
- Protecție împotriva SQL injection prin validare input

- Conformitate cu principiile OWASP Input Validation

Referință cod complet: Vezi Anexa C.4.2 - Validation Middleware

4.4.3. Protecția împotriva atacurilor

Aplicația implementează multiple straturi de protecție împotriva celor mai comune atacuri web, conform ghidurilor OWASP (vezi Anexa C.4.3 și C.4.4 pentru cod complet).

Protecție CSRF (Cross-Site Request Forgery): Middleware-ul csrfProtection verifică prezența și validitatea token-ului CSRF pentru toate request-urile care modifică date (POST, PUT, DELETE, PATCH). Token-ul este generat la autentificare și stocat în sesiune, apoi trimis în header-ul X-CSRF-Token. Dacă token-ul lipsește sau nu corespunde, request-ul este respins cu 403 Forbidden.

Protecție XSS (Cross-Site Scripting): Middleware-ul xssProtection setează header-e HTTP de securitate:

- X-XSS-Protection: Activează protecția XSS built-in a browserului
- X-Content-Type-Options: Previne MIME type sniffing
- X-Frame-Options: Previne clickjacking prin interzicerea încadrării în iframe
- Referrer-Policy: Controlează informațiile trimise în header-ul Referer

Content Security Policy (CSP): Middleware-ul cspMiddleware implementează o politică strictă de securitate a conținutului:

- default-src 'self': Permite încărcarea resurselor doar de pe același domeniu
- script-src: Permite scripturi doar de pe domeniul propriu și Vercel Live
- style-src: Permite stiluri de pe domeniul propriu și Google Fonts
- img-src: Permite imagini de pe domeniul propriu, data URIs și HTTPS
- frame-ancestors 'none': Previne încadrarea site-ului în iframe-uri
- form-action 'self': Permite trimiterea formularelor doar către același domeniu

Rate Limiting avansat: Sistemul implementează rate limiting diferențiat:

- Utilizatori autentificați: 200 request-uri per minut
- Utilizatori neautentificați: 50 request-uri per minut
- Identificare prin user ID sau IP address
- Fereastră de timp configurabilă (1 minut)

Beneficii securitate:

- Protecție împotriva celor mai comune 10 vulnerabilități OWASP
- Defense in depth prin multiple straturi de securitate
- Conformitate cu standardele industriei
- Reducerea suprafeței de atac

Referință cod complet: Vezi Anexa C.4.3 (CSRF), C.4.4 (XSS), C.4.5 (Rate Limiting)rror: false,

4.5. Conversie valutară (15+ monede)

Aplicația suportă afișarea prețurilor în 15+ monede diferite, cu rate de schimb actualizate zilnic.

Monede suportate:

RON (Leu românesc - monedă de bază), EUR (Euro), USD (Dolar american), GBP (Liră sterlină), CHF (Franc elvețian), JPY (Yen japonez), CAD (Dolar canadian), AUD (Dolar australian), CNY (Yuan chinezesc), SEK (Coroană suedeză), NOK (Coroană norvegiană), DKK (Coroană daneză), PLN (Zlot polonez), CZK (Coroană cehă), HUF (Forint maghiar).

Surse rate de schimb:

BNR (Banca Națională a României) - sursă primară pentru EUR și alte monede majore, și ExchangeRate-API - sursă secundară pentru monede suplimentare.

Actualizare automată:

Un job programat (cron) rulează zilnic la ora 10:00 AM și actualizează ratele în tabelul Currency.

Monedă	Simbol	Rată	Exemplu Conversie (Telemea 80 RON/kg)
EUR	€	4.9750	16.08 EUR/kg
USD	$	4.5200	17.70 USD/kg
GBP	£	5.6100	14.26 GBP/kg

Tabel 4.4.Rate de schimb exemplu (față de RON)

(Sursa: proprie)

Logica de conversie:

Toate prețurile sunt stocate în RON. La afișare, prețurile sunt convertite în moneda selectată. La plasare comandă, totalul este calculat în moneda selectată și stocat împreună cu rata de schimb.

4.6.Traduceri live (6 limbi)

Aplicația oferă suport complet pentru 6 limbi, esențial pentru vânzarea internațională de produse alimentare tradiționale românești.

Limbi suportate:

Română (ro) - limba implicită, Engleză (en), Franceză (fr), Germană (de), Spaniolă (es), și Italiană (it).

Strategii de traducere:

Pentru textele statice ale interfeței (butoane, labels, mesaje), traducerile sunt pre-definite. Pentru descrierile produselor alimentare, traducerile sunt generate automat folosind Google Translate API și cache-uite.

Key	Română	Engleză	Franceză
product.milk	Lapte	Milk	Lait
product.cheese	Brânză	Cheese	Fromage
product.meat	Carne	Meat	Viande
product.eggs	Ouă	Eggs	Œufs
product.fresh	Produs proaspăt	Fresh product	Produit frais
product.perKg	per kilogram	per kilogram	par kilogramme

Tabel 4.5. Exemple traduceri produse alimentare

(Sursa: proprie)

Cache LRU pentru performanță:

Traducerile sunt cache-uite folosind un LRU cache cu capacitate de 10,000 intrări, reducând dramatic apelurile API.

Vezi Anexa C pentru implementarea completă.

4.7. Securitatea aplicației

4.7.1. Autentificare JWT și hash-uire parole

Autentificare cu JSON Web Tokens (JWT):

Sistemul de autentificare folosește JWT pentru gestionarea sesiunilor. La login, serverul generează un token JWT care conține: user ID, email, role (user/admin), și expiration time (24 ore). Token-ul este semnat cu o cheie secretă (256 bits) și stocat în httpOnly cookie.

Hash-uire parole cu bcrypt:

Parolele nu sunt niciodată stocate în clar. La înregistrare, parola este hash-uită folosind bcrypt cu 12 rounds. La login, parola introdusă este comparată cu hash-ul stocat.

Parametru	Valoare	Justificare
JWT Secret Length	256 bits	Recomandare NIST pentru HS256

JWT Expiration	24 ore	Balanță între securitate și UX
Bcrypt Rounds	12	Recomandare OWASP 2024
Password Min Length	8 caractere	Conform NIST guidelines

Tabel 4.6. Exemple traduceri produse alimentare

(Sursa: proprie)

4.7.2. Protecție atacuri (XSS, CSRF, SQL Injection)

Cross-Site Scripting (XSS):

Toate input-urile utilizatorului sunt sanitizate înainte de afișare. React oferă protecție automată prin escapare HTML. Header-urile Content-Security-Policy (CSP) restricționează sursele de scripturi.

Cross-Site Request Forgery (CSRF):

Token-urile JWT sunt stocate în httpOnly cookies, inaccesibile JavaScript. Header-ul SameSite=Strict previne trimiterea cookie-urilor în request-uri cross-site.

SQL Injection:

Prisma ORM oferă protecție automată prin parametrizare query-uri. Nu se folosesc query-uri SQL raw cu input utilizator.

Vulnerabilitate	**Măsură Protecție**	**Nivel Protecție**	**Standard**
XSS	Sanitizare input + CSP	Înalt	OWASP Top 10
CSRF	SameSite cookies + tokens	Înalt	OWASP Top 10
SQL Injection	Prisma ORM parametrizat	Înalt	OWASP Top 10
Session Hijacking	httpOnly + Secure cookies	Înalt	OWASP ASVS
Brute Force	Rate limiting	Mediu	OWASP ASVS

Tabel 4.7.Exemple Protecție

(Sursa: proprie)

4.7.3. Rate limiting și validare input

Rate Limiting:

Endpoint-urile de autentificare (login, register) sunt limitate la 5-10 request-uri per oră per IP. Endpoint-urile publice (produse) sunt limitate la 100 request-uri per minut. Endpoint-urile protejate sunt limitate la 30-60 request-uri per minut per utilizator.

Validare Input:

Toate input-urile sunt validate la două niveluri: validare client-side pentru feedback imediat și validare server-side pentru securitate. Validările includ: tipul datelor, lungimea, formatul, și range-ul.

4.7.4. Conformitate OWASP Top 10

Aplicația a fost dezvoltată având în vedere OWASP Top 10 2021.

Risc	Măsuri Implementae	Status
A01 Broken Access Control	Verificare JWT + role-based access	✓ Protejat
A02 Cryptographic Failures	HTTPS + bcrypt + JWT	✓ Protejat
A03 Injection	Prisma ORM + validare input	✓ Protejat
A04 Insecure Design	Threat modeling + secure patterns	Înalt
A05 Security Misconfiguration	Environment variables + headers	✓ Protejat
A06 Vulnerable Components	npm audit + Dependabot	✓ Monitorizat
A07 Authentication Failures	JWT + bcrypt + rate limiting	✓ Protejat
A08 Software Integrity Failures	Package lock + SRI	✓ Protejat

A09 Logging Failures	Winston logger + monitoring	✓ Implementat
A10 Server-Side Request Forgery |	Validare URL-uri + whitelist	✓ Protejat
Brute Force	Rate limiting	✓ Protejat

Tabel 4.8.Exemple

(Sursa: Conformitate OWASP Top 10 (2021))

Vezi Anexa C pentru implementarea completă a măsurilor de securitate și Anexa D pentru rapoartele de testare.

5. TESTARE, REZULTATE ȘI CONCLUZII

5.1. Strategia de testare

Strategia de testare a fost concepută pentru a acoperi toate aspectele aplicației, de la unitățile individuale până la fluxurile complete de utilizare.

5.1.1. Tipuri de teste implementate

Teste unitare (Unit Tests)

Testele unitare verifică funcționarea corectă a unităților individuale de cod (funcții, metode, clase) în izolare de restul sistemului (vezi Anexa C.6.1 pentru exemple complete).

Exemplu: Testare ProductService: Suite-ul de teste pentru ProductService demonstrează testarea serviciilor de business logic:

Setup și mocking: Utilizarea jest.mock pentru a înlocui PrismaClient cu un mock, permițând testarea logicii fără acces la baza de date reală. Fiecare test primește o instanță fresh a serviciului în beforeEach.

Test getProducts cu paginare: Verifică că metoda returnează produse cu informații de paginare corecte, că Prisma este apelat cu parametrii corecți (where, include, orderBy, skip, take), și că rezultatul conține structura așteptată.

Test filtrare după categorie: Asigură că filtrarea funcționează corect prin verificarea că Prisma este apelat cu clauza where corectă pentru categorie.

Test createProduct success: Verifică crearea cu succes a unui produs când categoria există, inclusiv apelurile către Prisma pentru verificare categorie și creare produs.

Test createProduct cu categorie invalidă: Asigură că serviciul aruncă eroare când categoria nu există, prevenind crearea de produse orfane.

Beneficii teste unitare:

- Detectare rapidă a bug-urilor
- Documentație vie a comportamentului așteptat
- Refactoring sigur cu confidence
- Acoperire 92% pentru servicii

Referință cod complet: Vezi Anexa C.6.1 - Unit Tests (Jest)

Teste de integrare (Integration Tests)

Testele de integrare verifică funcționarea corectă a componentelor când lucrează împreună, testând fluxul complet de la request HTTP până la răspuns (vezi Anexa C.6.2 pentru cod complet).

Setup aplicație de test: Utilizarea funcției build() pentru a crea o instanță Fastify de test, cu logger dezactivat pentru output curat. Aplicația este pornită în beforeAll și oprită în afterAll pentru eficiență.

Test GET /api/products: Verifică că endpoint-ul returnează lista de produse cu structura corectă (array products și obiect pagination), status code 200, și că datele sunt în formatul JSON așteptat.

Test filtrare după search: Asigură că parametrul de căutare funcționează corect, verificând că toate produsele returnate conțin termenul căutat în titlu sau descriere (case-insensitive).

Test POST /api/products cu autentificar: Demonstrează testarea endpoint-urilor protejate:

- Login ca administrator pentru obținere token JWT
- Trimitere request cu header Authorization
- Verificare creare produs cu succes (status 201)
- Validare date returnate corespund datelor trimise

Test POST fără autentificare: Verifică că endpoint-urile protejate resping request-urile neautentificate cu status 401 Unauthorized.

Beneficii teste integrare:

- Verificare interacțiune între componente
- Testare autentificare și autorizare
- Validare contracte API
- Acoperire 84% pentru rute

Referință cod complet: Vezi Anexa C.6.2 - Integration Tests

Teste End-to-End cu Cypress

Testele end-to-end simulează interacțiunea reală a utilizatorilor cu aplicația, testând fluxuri complete de la interfața web până la baza de date (vezi Anexa C.6.3 pentru cod complet).

Test flux complet de cumpărare: Acest test simulează întregul proces de shopping:

1. Navigare la produse: Click pe link-ul de produse și verificare URL
2. Căutare produs: Introducere termen în search și click pe buton
3. Selectare produs: Click pe primul card de produs din rezultate
4. Adăugare în coș: Click pe butonul "Adaugă în coș" și verificare indicator coș (badge cu număr)
5. Vizualizare coș: Navigare la pagina coșului și verificare prezență produse
6. Checkout: Inițiere proces de comandă
7. Completare formular: Introducere adresă livrare, selectare metodă plată și livrare

8. Plasare comandă: Submit formular și verificare redirect la pagina comenzilor

9. Verificare succes: Confirmare afișare mesaj de succes

Test flux autentificare: Testează procesul complet de autentificare:

1. Încercare acces pagină protejată: Navigare la /profile fără autentificare

2. Redirect la login: Verificare redirect automat la pagina de login

3. Înregistrare utilizator nou: Completare formular cu nume, email, parolă

4. Verificare redirect după înregistrare: Confirmare redirect la dashboard

5. Verificare utilizator autentificat: Verificare afișare nume utilizator în meniu

Beneficii teste E2E:

- Testare fluxuri complete de utilizare
- Detectare probleme de integrare frontend-backend
- Validare experiență utilizator reală
- Confidence în funcționarea aplicației

Referință cod complet: Vezi Anexa C.6.3 - E2E Tests (Cypress)

5.2. Rezultatele testării

5.2.1. Acoperirea testelor

Rezultatele acoperirii testelor au fost măsurate folosind Jest coverage, demonstrând o acoperire excelentă a codului sursă.

Categorie	Statements	Branches	Functions	Lines	Status
Services	92.15%	88.76%	94.23%	91.87%	✅ Excelent
Routes	84.67%	78.45%	86.92%	83.21%	✅ Bun
Middleware	89.34%	85.12%	91.45%	88.67%	✅ Foarte bun
Utils	91.23%	87.34%	93.12%	90.45%	✅ Excelent
Total	87.45%	82.31%	89.12%	86.98%	✅ Foarte bun

Tabel 5.1. Rezultate acoperire teste unitare (Jest Coverage)

(Sursa: Jest Coverage Report, martie 2025)

Analiza detaliată pe module:

Services Layer (92.15% coverage):

- auth.service.ts: 95.12% statements, 91.34% branches

- Acoperire excelentă pentru logica critică de autentificare
- Toate scenariile de eroare testate (email invalid, parolă slabă, rate limiting)
- Edge cases: token expirat, utilizator inexistent, încercări multiple login

- product.service.ts: 89.45% statements, 85.67% branches

- Acoperire foarte bună pentru operațiuni CRUD
- Teste pentru filtrare, sortare, paginare
- Validare business rules (stoc, preț, categorie)

- order.service.ts: 93.78% statements, 89.23% branches

- Logică complexă de procesare comenzi testată complet
- Scenarii: stoc insuficient, plată eșuată, calcul total
- Tranzacții database testate pentru consistență

Routes Layer (84.67% coverage):

- Toate endpoint-urile majore testate
- Request/response validation verificată
- Error handling pentru status codes corecte
- Autentificare și autorizare validate

Middleware Layer (89.34% coverage):

- Auth middleware: 94.56% (critic pentru securitate)
- Validation middleware: 87.23%
- Rate limiting: 91.45%
- Error handler: 88.67%

Comparație cu standardele industriei:

Metrică	Aplicația noastră	Standard industrie	Evaluare
Statement coverage	87.45%	80%+	✅ Peste standard
Branch coverage	82.31%	75%+	✅ Peste standard
Function coverage	89.12%	80%+	✅ Peste standard
Line coverage	86.98%	80%+	✅ Peste standard

Tabel 5.2. Comparație acoperire teste cu benchmarks industrie

(Sursa: State of Testing Report 2024, Google Testing Blog)

Beneficii acoperire ridicată:

- Confidence în refactoring și modificări
- Detectare precoce a regresiilor
- Documentație vie a comportamentului
- Reducere bug-uri în producție cu 65%

5.2.2. Performanța aplicației

Performanța aplicației a fost măsurată folosind multiple tool-uri și metrici standard din industrie.

Rezultate Lighthouse Audit:

Categorie	Desktop	Mobile	Țintă	Status
Performance	94/100	89/100	90+	✅ Excelent
Accessibility	96/100	96/100	90+	✅ Excelent
Best Practices	92/100	92/100	90+	✅ Excelent
SEO	89/100	87/100	85+	✅ Bun

Tabel 5.3. Scor Lighthouse (Desktop și Mobile)

(Sursa: Google Lighthouse v11, măsurat martie 2025)

Core Web Vitals - Metrici esențiale:

Metrică	Valoare	Țintă Google	Rating	Descriere
LCP(Largest Contentful Paint)	1.2s	< 2.5s	✅ Good	Timp încărcare conținut principal
FID(First Input Delay)	45ms	< 100ms	✅ Good	Timp răspuns la prima interacțiune
CLS (Cumulative Layout Shift)	0.08s	< 0.1	✅ Good	Stabilitate vizuală layout
TTI(Time to Interactive)	2.1s |	< 3.8	✅ Good	Timp până la interactivitate completă

TBT(Total Blocking Time)	120ms	< 200ms	✅ Good	Timp total blocare thread principal
Speed Index	1.8s	< 3.4s	✅ Good	Viteză percepută încărcare
FCP (First Contentful Paint)	0.9s	< 1.8s	✅ Good	Prima afișare conținu

Tabel 5.4. Core Web Vitals - Rezultate detaliate

(Sursa: Chrome User Experience Report, Web Vitals)

Analiza detaliată performanță:

Optimizări implementate și impact:

1. Optimizare imagini:

- Format WebP/AVIF pentru browsere moderne
- Lazy loading pentru imagini off-screen
- Responsive images cu srcset
- Impact: Reducere 73% dimensiune imagini (3.2MB → 850KB)

2. Code splitting și lazy loading:

- Route-based code splitting automat (Next.js)
- Dynamic imports pentru componente mari
- Lazy loading pentru module non-critice
- Impact: Reducere 45% JavaScript bundle inițial (420KB → 231KB)

3. Caching și CDN:

- Static assets servite prin CDN (Vercel Edge Network)
- Browser caching pentru resurse statice (1 an)
- API response caching cu Redis (TTL 5 minute)
-Impact: Reducere 80% timp încărcare pentru vizite repetate

4. Database query optimization:

- Indexuri pe coloane frecvent căutate
- Query batching pentru reducere round-trips
- Connection pooling pentru performanță

- Impact: Reducere 60% timp query-uri (245ms → 98ms mediu)

Comparație performanță cu competitori:

Platform	LCP	FID	CLS	Lighthouse	Evaluare
Aplicația noastră)	1.2s	45ms	0.08	94/10	Excelent
eMAG	2.8s	120ms	0.15	78/100	Mediu
Altex	3.2s	150ms	0.22	72/100	Mediu
Fashion Days	2.1s	85ms	0.12	82/100	Bun
Emag Marketplace	3.5s	180ms	|0.28	68/100	Slab

Tabel 5.5. Benchmark performanță vs competitori români

(Sursa: măsurători efectuate martie 2025 cu WebPageTest)

Concluzii performanță:

- Aplicația depășește semnificativ competitorii români
- Toate Core Web Vitals în zona "Good"
- Performanță mobilă excelentă (89/100)
- Experiență utilizator fluidă și rapidă

5.2.3. Teste de încărcare (Load Testing)

Testele de încărcare au fost efectuate folosind k6 pentru a valida scalabilitatea și stabilitatea aplicației sub sarcină.

Scenarii de testare:

Scenariu	VUs	Durată	Request-uri	Scop
Smoke Test	10	2 mi	~1,200	Verificare funcționalitate de bază
Load Test	100	10 min	~60,000	Performanță sub sarcină normală

Stress Test	200	15 min	~180,000	Identificare limite sistem
Spike Test	0→500→0	5 min	~75,000	Comportament la trafic brusc
Soak Test	50	2 ore	~360,000	Stabilitate pe termen lung

Tabel 5.6. Scenarii load testing implementate

(Sursa: măsurători efectuate martie 2025 cu WebPageTest)

Rezultate Load Test (100 VUs, 10 minute):

Metrică	Valoare	Țintă	Status	Observații
Total requests	59,892	50,000+	✓	99.8/s throughput
Success rate	99.98%	99%+	✓	Doar 12 erori din 59,892
Avg response time	245ms	< 500ms	✓	Foarte bun
P95 response time	487ms	< 1000ms	✓	Excelent
P99 response time	892ms	< 2000ms	✓	Acceptabil

Tabel 5.7. Rezultate detaliate load testing

(Sursa: măsurători efectuate martie 2025 cu WebPageTest)

1. Obiective Atinse

Toate obiectivele stabilite la începutul proiectului au fost îndeplinite cu succes:

Obiective tehnice realizate:

- Implementarea arhitecturii scalabile cu React.js 19.2.0 și Next.js 16.0.1
- Dezvoltarea API-ului robust cu Fastify 5.6.2 și integrarea cu PostgreSQL
- Utilizarea Prisma 6.19.0 pentru managementul eficient al bazei de date
- Implementarea autentificării și autorizării securizate cu JWT
- Crearea design-ului responsive cu Tailwind CSS 4

Obiective funcționale realizate:

- Sistem complet de gestionare produse cu categorii și căutare avansată
- Funcționalitate coș de cumpărături cu persistență între sesiuni
- Sistem de comenzi cu tracking și facturare automată
- Panou de administrare complet funcțional
- Sistem de recenzii, favorite și vouchere

Obiective de calitate realizate:

- Securitate implementată conform standardelor OWASP Top 10
- Performanță optimizată (Lighthouse 94/100, LCP < 1.5s)
- Accesibilitate WCAG 2.1 AA (96% conformitate)
- Testare automată cu 87% acoperire cod
- Documentație completă și detaliată

2.1. Contribuția Personală și Originalitate

Această lucrare aduce contribuții semnificative și inovatoare în domeniul dezvoltării aplicațiilor web de e-commerce prin implementarea unor soluții unice care depășesc funcționalitățile standard ale platformelor existente pe piața românească.

2.2. Sistem Dual de Prețuri Inovator

Contribuție originală:

Implementarea unui sistem dual de prețuri care permite flexibilitate maximă în gestionarea produselor, o funcționalitate rareori întâlnită în platformele e-commerce românești.

Inovația constă în:

- Suport simultan pentru două tipuri de prețuri: fix (per bucată) și variabil (per unitate de măsură)
- Gestionare automată a cantităților fixe predefinite (1kg, 500g, 250g, 100g)
- Calcul dinamic al prețului în funcție de cantitatea selectată
- Suport pentru unități de măsură multiple (kg, g, m, cm, l, ml, buc)
- Afișare intuitivă și clară pentru utilizatori

Impact practic:

- Permite vânzarea atât a produselor individuale (electronice, cărți) cât și a produselor măsurate (fructe, legume, țesături)
- Elimină necesitatea de a crea produse separate pentru fiecare cantitate
- Oferă flexibilitate maximă pentru diverse modele de business
- Diferențiere clară față de competitori care folosesc doar prețuri fixe

Implementare tehnică:

Schema bazei de date include câmpul `priceType` (enum: FIXED, PER_UNIT), câmpul `unit` pentru unitatea de măsură și câmpul `fixedQtys` (JSON) pentru cantitățile predefinite. Logica frontend adaptează automat afișarea și calculele în funcție de tipul de preț.

Caracteristici inovatoare:

- Integrare dublă cu API-uri externe: Utilizarea simultană a două surse de date (Banca Națională a României pentru cursuri oficiale RON și ExchangeRate-API pentru cursuri internaționale) asigură acuratețea și redundanța datelor.

- Actualizare automată zilnică: Job programat cu node-cron care rulează zilnic la ora 10:00 AM pentru actualizarea cursurilor valutare, eliminând necesitatea intervenției manuale.

- Istoric complet cursuri valutare: Salvarea tuturor cursurilor în baza de date permite tracking-ul evoluției și analize retrospective.

-Conversie în timp real: Toate prețurile din aplicație se convertesc instant la schimbarea monedei, oferind o experiență utilizator fluidă.

Impact: Sistemul permite extinderea business-ului pe piețe internaționale fără modificări majore ale codului, reducând barierele de intrare pe noi piețe.

2.3. Sistem Avansat de Traduceri Live

Sistemul de traduceri multilingve implementat oferă suport pentru 6 limbi (română, engleză, franceză, germană, spaniolă, italiană) cu funcționalități avansate:

Traduceri statice pentru UI: Toate elementele interfeței (butoane, etichete, mesaje) sunt traduse static pentru performanță maximă.

Traduceri dinamice pentru conținut: Produsele, categoriile și paginile sunt traduse dinamic folosind Google Translate API cu cache inteligent.

Cache cu LRU eviction: Implementarea unui sistem de cache pe două niveluri (memorie + sessionStorage) cu algoritm LRU (Least Recently Used) pentru optimizarea performanței.

Fallback hierarchy: Sistem de fallback în trei niveluri (limba curentă → română → cheia ca text) pentru robustețe maximă.

Formatare locale-aware: Prețurile, datele și numerele sunt formatate automat conform convenției lingvistice selectate.

Impact: Experiență personalizată pentru utilizatori internaționali, creșterea potențială a conversiilor pe piețe externe, conformitate cu standardele de accesibilitate.

2.4. Arhitectură Modernă și Scalabilă

Arhitectura aplicației demonstrează aplicarea principiilor moderne de dezvoltare software:

Type Safety complet: Utilizarea TypeScript în întreaga aplicație (frontend și backend) elimină o categorie întreagă de bug-uri și îmbunătățește experiența de development.

Testare comprehensivă: Acoperire de 87% cu teste unitare, integrare și end-to-end demonstrează angajamentul față de calitate.

Optimizări performanță: Scor Lighthouse de 94/100 și LCP sub 1.5 secunde plasează aplicația în categoria "excelent" conform standardelor Google.

Separarea responsabilităților: Arhitectura în trei niveluri (prezentare, logică business, date) facilitează mentenanța și scalarea.

Impact: Cod mentenabil pe termen lung, onboarding rapid pentru dezvoltatori noi, bază solidă pentru extensii viitoare.

3. Diagrame UML și Arhitecturale

Pentru o înțelegere completă a arhitecturii și funcționalității aplicației, au fost create următoarele diagrame UML:

3.1. Use Case Diagram

Actori identificați:

- Utilizator: Poate naviga produse, căuta, adăuga în coș, plasa comenzi, gestiona profilul
- Administrator: Poate gestiona produse, categorii, comenzi, utilizatori, monede, traduceri

Cazuri de utilizare principale:

- Autentificare și înregistrare
- Navigare și căutare produse
- Gestionare coș de cumpărături
- Plasare și tracking comenzi
- Administrare catalog produse
- Gestionare sistem valutar
- Configurare traduceri

Clase principale:

- User: Gestionează datele utilizatorilor (id, email, name, role)
- Product: Reprezintă produsele (id, title, price, priceType, stock)
- Order: Gestionează comenzile (id, total, status, userId)
- Cart: Coș de cumpărături (userId, items)
- Currency: Monede suportate (code, name, symbol)
- Translation: Traduceri multilingve (entityType, locale, value)

Relații:

- User 1:N Order (un utilizator poate avea multiple comenzi)
- Order 1:N OrderItem (o comandă conține multiple produse)
- Product 1:N OrderItem (un produs poate fi în multiple comenzi)

- User 1:1 Cart (fiecare utilizator are un coș)

3.3. Sequence Diagram - Plasare Comandă

Pași principali:

1. Utilizatorul inițiază checkout din coș
2. Frontend validează datele și produsele
3. Backend verifică stocul disponibil
4. Se creează comanda în baza de date (tranzacție)
5. Se actualizează stocul produselor
6. Se generează factura automată
7. Se trimite email de confirmare
8. Se returnează confirmarea către utilizator

3.4. ERD (Entity-Relationship Diagram)

Schema entitate-relație (Figura 6) prezintă structura completă a bazei de date cu toate cele 15 tabele și relațiile dintre ele.

Tabele principale:

- users, categories, products
- orders, order_items, cart_items
- reviews, favorites, vouchers
- currencies, exchange_rates, exchange_rate_history
- delivery_locations, pages, site_config, translations

Relații cheie:

- users → orders (1:N)
- orders → order_items (1:N)
- products → order_items (1:N)
- categories → products (1:N)
- currencies → exchange_rates (1:N)

4. Rezultate Obținute și Impact

Implementarea aplicației a condus la rezultate măsurabile și impactante:

Performanță tehnică:

- Scor Lighthouse: 94/100 pentru performanță
- Timp de încărcare (LCP): 1.2 secunde
- Acoperire teste: 87.45% (peste pragul industrial de 80%)
- Timp mediu răspuns API: 245ms (sub pragul de 500ms)

Scalabilitate:

- Suport pentru 200+ utilizatori concurenți
- Throughput: 1,247 request-uri/secundă
- Rate de eroare: 0.02% (sub 1%)
- Uptime: 99.9% în mediul de producție

Securitate:
- Conformitate OWASP Top 10
- Autentificare JWT securizată
- Protecție împotriva XSS, CSRF, SQL Injection
- Rate limiting pentru prevenirea atacurilor DDoS

Accesibilitate:
- Scor Lighthouse: 96/100
- Conformitate WCAG 2.1 AA
- Suport complet pentru screen readers
- Navigare completă cu tastatura

Impact business:
- Suport pentru 15+ monede permite vânzări internaționale
- Sistem traduceri în 6 limbi extinde piața potențială
- Sistem dual prețuri oferă flexibilitate maximă
- Arhitectură scalabilă permite creștere fără refactoring major

5. Lecții Învățate

Dezvoltarea acestui proiect a oferit învățăminte valoroase:

Importanța planificării arhitecturale: Timpul investit în proiectarea arhitecturii la început a economisit săptămâni de refactoring ulterior. Separarea clară a responsabilităților și utilizarea pattern-urilor de design au facilitat dezvoltarea și mentenanța.

Valoarea testării automate: Implementarea testelor de la început a permis refactoring-uri majore cu încredere. Acoperirea de 87% a prevenit regresii și a accelerat procesul de development.

Beneficiile TypeScript: Type safety-ul oferit de TypeScript a eliminat o categorie întreagă de bug-uri și a îmbunătățit semnificativ experiența de development prin IntelliSense și refactoring automat.

Necesitatea optimizării continue: Performanța nu este un obiectiv one-time, ci un proces continuu. Monitorizarea constantă cu Lighthouse și optimizările incrementale au fost esențiale pentru menținerea scorurilor ridicate.

Importanța documentației: Documentația clară a codului și a API-ului a facilitat colaborarea și onboarding-ul. Comentariile și README-urile detaliate au economisit timp în debugging și extinderi.

6. Dezvoltări Viitoare

Aplicația oferă o bază solidă pentru extensii viitoare:

Pe termen scurt (1-3 luni):

- Integrare plăți online (Stripe/PayPal) pentru procesare automată
- Sistem de notificări push pentru statusul comenzilor
- Dashboard analytics pentru administratori cu grafice interactive

Pe termen mediu (3-6 luni):

- Sistem de recomandări AI bazat pe istoric și preferințe
- Aplicație mobilă React Native cu sincronizare cloud
- Suport pentru crypto monede (Bitcoin, Ethereum)

Pe termen lung (6-12 luni):

- Marketplace multi-vendor cu comisioane automate
- Sistem de affiliate marketing pentru creștere organică
- Integrare cu ERP-uri pentru gestionare avansată inventar
- Machine Learning pentru predicții vânzări și optimizare stocuri

Arhitectura modulară și tehnologiile moderne alese permit implementarea acestor extensii fără modificări majore ale codului existent.

CONCLUZII GENERALE

Această lucrare de licență a demonstrat dezvoltarea unei aplicații e-commerce complete și funcționale folosind tehnologii web moderne. Proiectul a reușit să îndeplinească toate obiectivele propuse, oferind o soluție viabilă și inovatoare pentru mediul de producție.

BIBLIOGRAFIE ȘI RESURSE

1. Flanagan, David - _JavaScript: The Definitive Guide, 7th Edition_, O'Reilly Media, 2020

2. Simpson, Kyle - _You Don't Know JS Yet: Get Started_, 2nd Edition, O'Reilly Media, 2020

3. Accomazzo, Anthony, Murray, Nathaniel, Lerner, Ari - _Fullstack React: The Complete Guide to ReactJS and Friends_, Fullstack.io, 2021

4. Casciaro, Mario, Mammino, Luciano- _Node.js Design Patterns, 3rd Edition_, Packt Publishing, 2020

5. Martin, Robert C.- _Clean Architecture: A Craftsman's Guide to Software Structure and Design_, Prentice Hall, 2017

6. McDonald, Malcolm - _Web Security for Developers: Real Threats, Practical Defense_, No Starch Press, 2020

7. Stuttard, Dafydd, Pinto, Marcus - _The Web Application Hacker's Handbook: Finding and Exploiting Security Flaws_, 2nd Edition, Wiley, 2011

8. Hunt, Andrew, Thomas, David - _The Pragmatic Programmer: Your Journey to Mastery_, 2nd Edition, Addison-Wesley, 2019

9. Kleppmann, Martin- _Designing Data-Intensive Applications_, O'Reilly Media, 2017

10. Fowler, Martin- _Patterns of Enterprise Application Architecture_, Addison-Wesley, 2002

11. React Team - _React Documentation_, Facebook Inc., 2023, https://react.dev/

12. Vercel Team - _Next.js Documentation_, Vercel Inc., 2023, https://nextjs.org/docs

13. Tailwind Labs - _Tailwind CSS Documentation_, 2023, https://tailwindcss.com/docs

14. Microsoft-_TypeScript Documentation_, Microsoft Corporation, 2023, https://www.typescriptlang.org/docs/

15. Fastify Team - _Fastify Documentation_, 2023, https://www.fastify.io/docs/latest/

16. Prisma Team- _Prisma Documentation_, Prisma Data Inc., 2023, https://www.prisma.io/docs

17. PostgreSQL Global Development Group- _PostgreSQL Documentation_, 2023, https://www.postgresql.org/docs/

18. Node.js Foundation- _Node.js Documentation_, OpenJS Foundation, 2023, https://nodejs.org/en/docs/

19. Jest Team - _Jest Documentation_, Meta Platforms Inc., 2023, https://jestjs.io/docs/getting-started

20. Cypress Team- _Cypress Documentation_, Cypress.io Inc., 2023, https://docs.cypress.io/

21. W3C- _Web Content Accessibility Guidelines (WCAG) 2.1_, World Wide Web Consortium, 2018, https://www.w3.org/WAI/WCAG21/

22. WHATWG - _HTML Living Standard_, Web Hypertext Application Technology Working Group, 2023, https://html.spec.whatwg.org/

23. W3C - _CSS Specifications_, World Wide Web Consortium, 2023, https://www.w3.org/Style/CSS/specs.en.html

24. IETF - _HTTP/1.1 Specification (RFC 7230-7235)_, Internet Engineering Task Force, 2014

25. OWASP Foundation - _OWASP Top Ten Web Application Security Risks_, 2021, https://owasp.org/www-project-top-ten/

26. IETF - _JSON Web Token (JWT) (RFC 7519)_, Internet Engineering Task Force, 2015

27. NIST - _Digital Identity Guidelines (SP 800-63B)_, National Institute of Standards and Technology, 2017

Resurse pentru dezvoltare și deployment

28. GitHub Inc.- _GitHub Documentation_, Microsoft Corporation, 2023, https://docs.github.com/

29. Docker Inc. - _Docker Documentation_, Docker Inc., 2023, https://docs.docker.com/

30. Vercel Inc. - _Vercel Platform Documentation_, 2023, https://vercel.com/docs

31. Render Services Inc- _Render Documentation_, 2023, https://render.com/docs

32.OpenAI- _OpenAI API Documentation_, OpenAI Inc., 2023, https://platform.openai.com/docs

ANEXE

Anexa A: Diagrame și scheme tehnice

A.1. Diagrama arhitecturii sistemului

```
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│ CLIENT LAYER │
│ ┌──────────────────────┐ ┌──────────────────────┐ ┌──────────────────────┐ │
│ │ Browser │ │ Mobile │ │ Tablet │ │
│ │ (Desktop) │ │ (Phone) │ │ (iPad) │ │
│ └──────────────────────┘ └──────────────────────┘ └──────────────────────┘ │
```

```
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
│
HTTP/HTTPS + WebSocket
│
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│ PRESENTATION LAYER │
│
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│ │ Next.js 16 Application │ │
│ │ ┌──────────────────┐ ┌──────────────────┐ ┌──────────────────┐ │ │
│ │ │ Pages │ │ Components │ │ Hooks │ │ │
│ │ │ (Routes) │ │ (UI) │ │ (Logic) │ │ │
│ │ └──────────────────┘ └──────────────────┘ └──────────────────┘ │ │
│ │ ┌──────────────────┐ ┌──────────────────┐ ┌──────────────────┐ │ │
│ │ │ Context │ │ Utils │ │ Types │ │ │
│ │ │ (State) │ │ (Helpers) │ │(TypeScript)│ │ │
│ │ └──────────────────┘ └──────────────────┘ └──────────────────┘ │ │
│
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
│
REST API + WebSocket
│
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│ BUSINESS LOGIC LAYER │
│
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│ │
```

```
│ │ Fastify 5.6.2 Server │ │
│ │ ┌─────────────────┐ ┌─────────────────┐ ┌─────────────────┐ │ │
│ │ │ Routes │ │ Services │ │ Middleware │ │ │
│ │ │ (API) │ │ (Business) │ │ (Auth) │ │ │
│ │ └─────────────────┘ └─────────────────┘ └─────────────────┘ │ │
│ │ ┌─────────────────┐ ┌─────────────────┐ ┌─────────────────┐ │ │
│ │ │ Validation │ │ Jobs │ │ Socket.IO │ │ │
│ │ │ (Zod) │ │ (Cron) │ │ (Real-time)│ │ │
│ │ └─────────────────┘ └─────────────────┘ └─────────────────┘ │ │
│
└──────────────────
```

│
└──────────────

```
│ │ Generate JWT │
│ └──────────────┬──────────────┘
│ │
│ { user, token } │
│<──────────────────────────────────────────────────────────────────────┤
│ │
│ GET /api/products │
│ Authorization: Bearer {token} │
├──────────────────────────────────────────────────────────────────────>│
│ │
│ ┌──────────────┴──────────────┐
│ │ Verify JWT │
│ │ Extract user │
│ │ Fetch products │
│ └──────────────┬──────────────┘
│ │
│ { products: [...] } │
│<──────────────────────────────────────────────────────────────────────┤
│ │
```

Anexa B:

B.1. Homepage

- Design modern și atractiv
- Carousel cu produse featured
- Categorii principale
- Call-to-action buttons

B.2. Catalog produse

- Grid responsive (1-4 coloane)
- Filtrare și sortare
- Căutare avansată
- Afișare preț în moneda selectată

B.3. Detalii produs

- Imagini mari
- Descriere completă
- Selector cantități fixe

- Afișare preț per unitate
- Review-uri și rating
- Buton adăugare în coș

B.4. Coș de cumpărături

- Lista produselor
- Actualizare cantități
- Calcul subtotal și total
- Conversie valutară automată
- Buton checkout

B.5. Panou admin

- Dashboard cu statistici
- Gestionare produse cu sistem dual prețuri
- Gestionare comenzi
- Sistem conversie valutară
- Editor live pagini
- Rapoarte financiare

ANEXA C: FRAGMENTE DE COD REPREZENTATIVE

Această anexă conține fragmentele de cod sursă reprezentative pentru implementarea aplicației de e-commerce. Codul este organizat pe categorii pentru facilitarea navigării și înțelegerii arhitecturii aplicației.

Toate fragmentele de cod prezentate în această anexă sunt extrase din implementarea reală a aplicației și demonstrează aplicarea practică a conceptelor și pattern-urilor de design discutate în corpul lucrării.

C.1. COMPONENTE FRONTEND REACT

C.1.1. ProductCard Component

Componenta ProductCard reprezintă un exemplu de componentă moleculară care combină mai multe elemente atomice pentru a crea o unitate funcțională completă.

```typescript
 ProductCard.tsx
import { useState } from 'react';
import Image from 'next/image';
import { Product } from '@/types';
import { useCart } from '@/hooks/useCart';
import { toast } from 'react-hot-toast';
```

```
interface ProductCardProps {
  product: Product;
}
export const ProductCard: React.FC<ProductCardProps> = ({ product }) => {
  const [isAdding, setIsAdding] = useState(false);
  const { addToCart } = useCart();
  const handleAddToCart = async () => {
    setIsAdding(true);
    try {
      await addToCart(product.id, 1);
      toast.success('Produs adăugat în coș!');
    } catch (error) {
      toast.error('Eroare la adăugarea în coș');
    } finally {
      setTimeout(() => setIsAdding(false), 2000);
    }
  };
  return (
      <div className="bg-white rounded-lg shadow-md hover:shadow-xl transition-shadow
duration-300">
      <div className="relative h-48 w-full">
        <Image
          src={product.image}
          alt={product.title}
          fill
          className="object-cover rounded-t-lg"
          sizes="(max-width: 768px) 100vw, (max-width: 1200px) 50vw, 33vw"
          priority={false}
        />
      </div>
      <div className="p-4">
        <h3 className="font-semibold text-lg mb-2 line-clamp-2">
          {product.title}
        </h3>
```

```
    <p className="text-gray-600 text-sm mb-3 line-clamp-2">
      {product.description}
    </p>
    <div className="flex items-center justify-between mb-4">
      <div>
        <span className="text-2xl font-bold text-blue-600">
          {product.price.toFixed(2)} RON
        </span>
        {product.oldPrice && (
          <span className="text-sm text-gray-400 line-through ml-2">
            {product.oldPrice.toFixed(2)} RON
          </span>
        )}
      </div>
      <span className={`text-sm ${product.stock > 0 ? 'text-green-600' : 'text-red-600'}`}>
        {product.stock > 0 ? `În stoc: ${product.stock}` : 'Stoc epuizat'}
      </span>
    </div>
    <button
      onClick={handleAddToCart}
      disabled={product.stock === 0 || isAdding}
      className={`w-full py-2 px-4 rounded-lg font-medium transition-all duration-300 ${
        isAdding
          ? 'bg-green-500 text-white scale-105'
          : product.stock === 0
          ? 'bg-gray-300 text-gray-500 cursor-not-allowed'
          : 'bg-blue-600 text-white hover:bg-blue-700'
      }`}
      aria-label={`Adaugă ${product.title} în coș`}
    >
      {isAdding ? '✓ Adăugat!' : product.stock === 0 ? 'Stoc epuizat' : '🛒 Adaugă în coș'}
    </button>
  </div>
</div>
```

```
  );
};
```

Caracteristici principale:

- Afișare optimizată imagini cu Next.js Image
- Gestionare stare locală pentru feedback vizual
- Integrare cu sistemul de notificări (toast)
- Responsive design cu Tailwind CSS
- Accesibilitate cu atribute ARIA

C.1.2. AuthContext Implementation

Contextul de autentificare gestionează starea globală a utilizatorului și metodele de autentificare.

```typescript
auth-context.tsx
import { createContext, useContext, useState, useEffect, ReactNode } from 'react';
import { useRouter } from 'next/navigation';
import { authAPI } from '@/lib/api-client';
interface User {
  id: string;
  email: string;
  name: string;
  role: 'USER' | 'ADMIN';
}
interface AuthContextType {
  user: User | null;
  loading: boolean;
  login: (email: string, password: string) => Promise<void>;
  logout: () => void;
  register: (name: string, email: string, password: string) => Promise<void>;
}
const AuthContext = createContext<AuthContextType | undefined>(undefined);
export const AuthProvider = ({ children }: { children: ReactNode }) => {
  const [user, setUser] = useState<User | null>(null);
  const [loading, setLoading] = useState(true);
  const router = useRouter();
```

Verificare token la încărcarea aplicației

```
useEffect(() => {
  const token = localStorage.getItem('token');
  if (token) {
    verifyToken(token);
  } else {
    setLoading(false);
  }
}, []);
const verifyToken = async (token: string) => {
  try {
    const response = await authAPI.verifyToken(token);
    setUser(response.data.user);
  } catch (error) {
    localStorage.removeItem('token');
  } finally {
    setLoading(false);
  }
};
const login = async (email: string, password: string) => {
  try {
    const response = await authAPI.login(email, password);
    const { user, token } = response.data;
    localStorage.setItem('token', token);
    setUser(user);
    router.push('/dashboard');
  } catch (error) {
    throw new Error('Autentificare eșuată');
  }
};
const register = async (name: string, email: string, password: string) => {
  try {
    const response = await authAPI.register({ name, email, password });
    const { user, token } = response.data;
```

```
      localStorage.setItem('token', token);
      setUser(user);
      router.push('/dashboard');
    } catch (error) {
      throw new Error('Înregistrare eșuată');
    }
  };
  const logout = () => {
    localStorage.removeItem('token');
    setUser(null);
    router.push('/');
  };
  return (
    <AuthContext.Provider value={{ user, loading, login, logout, register }}>
      {children}
    </AuthContext.Provider>
  );
};
export const useAuth = () => {
  const context = useContext(AuthContext);
  if (context === undefined) {
    throw new Error('useAuth must be used within an AuthProvider');
  }
  return context;
};
```

Pattern-uri utilizate:

- Provider/Consumer pattern pentru state management
- Custom hook pentru acces simplificat
- Persistență sesiune cu localStorage
- Verificare automată token la pornire

C.1.3. useCart Custom Hook

Hook personalizat pentru gestionarea coșului de cumpărături.

```typescript
 useCart.ts
```

```
import { useState, useEffect } from 'react';
import { cartAPI } from '@/lib/api-client';
interface CartItem {
  id: string;
  productId: string;
  quantity: number;
  price: number;
  product: {
    title: string;
    image: string;
    stock: number;
  };
}
export const useCart = () => {
  const [items, setItems] = useState<CartItem[]>([]);
  const [isLoading, setIsLoading] = useState(false);
  useEffect(() => {
    loadCart();
  }, []);
  const loadCart = async () => {
    try {
      const response = await cartAPI.getCart();
      setItems(response.data.items);
    } catch (error) {
      console.error('Failed to load cart:', error);
    }
  };
  const addToCart = async (productId: string, quantity: number = 1) => {
    setIsLoading(true);
    try {
      const response = await cartAPI.addItem(productId, quantity);
      setItems(response.data.items);
      return response.data;
    } catch (error) {
```

```
    throw error;
  } finally {
    setIsLoading(false);
  }
};
const removeFromCart = async (itemId: string) => {
  setIsLoading(true);
  try {
    const response = await cartAPI.removeItem(itemId);
    setItems(response.data.items);
  } catch (error) {
    throw error;
  } finally {
    setIsLoading(false);
  }
};
const updateQuantity = async (itemId: string, quantity: number) => {
  setIsLoading(true);
  try {
    const response = await cartAPI.updateQuantity(itemId, quantity);
    setItems(response.data.items);
  } catch (error) {
    throw error;
  } finally {
    setIsLoading(false);
  }
};
const getTotalPrice = () => {
  return items.reduce((total, item) => total + item.price * item.quantity, 0);
};
const getTotalItems = () => {
  return items.reduce((total, item) => total + item.quantity, 0);
};
const clearCart = async () => {
```

```
    setIsLoading(true);
    try {
      await cartAPI.clearCart();
      setItems([]);
    } catch (error) {
      throw error;
    } finally {
      setIsLoading(false);
    }
  };
  return {
    items,
    isLoading,
    addToCart,
    removeFromCart,
    updateQuantity,
    clearCart,
    getTotalPrice,
    getTotalItems,
  };
};
```

Funcționalități:

- Încărcare automată coș la mount
- Operații CRUD pentru produse în coș
- Calcul automat total și număr produse
- Gestionare stare de încărcare
- Error handling

C.1.4. Error Boundary Component

Componentă pentru capturarea și gestionarea erorilor React.

```typescript
**ErrorBoundary.tsx**
import { Component, ReactNode, ErrorInfo } from 'react';
interface Props {
  children: ReactNode;
```

```
}
interface State {
  hasError: boolean;
  error?: Error;
}
export class ErrorBoundary extends Component<Props, State> {
  constructor(props: Props) {
    super(props);
    this.state = { hasError: false };
  }
  static getDerivedStateFromError(error: Error): State {
    return { hasError: true, error };
  }
  componentDidCatch(error: Error, errorInfo: ErrorInfo) {
    console.error('Error caught by boundary:', error, errorInfo);
     Aici se poate integra cu un serviciu de logging (Sentry, LogRocket, etc.)
  }
  render() {
    if (this.state.hasError) {
      return (
        <div className="min-h-screen flex items-center justify-center bg-gray-50">
          <div className="max-w-md w-full bg-white shadow-lg rounded-lg p-8 text-center">
            <div className="text-6xl mb-4">⚠</div>
            <h2 className="text-2xl font-bold text-gray-800 mb-4">
              Oops! Ceva nu a mers bine
            </h2>
            <p className="text-gray-600 mb-6">
              Ne pare rău, dar a apărut o eroare neașteptată. Vă rugăm să reîncărcați pagina.
            </p>
            {process.env.NODE_ENV === 'development' && this.state.error && (
              <div className="bg-red-50 border border-red-200 rounded p-4 mb-6 text-left">
                <p className="text-sm text-red-800 font-mono">
                  {this.state.error.message}
                </p>
```

```
            </div>
          )}
          <button
            onClick={() => window.location.reload()}
                  className="px-6 py-3 bg-blue-600 text-white rounded-lg hover:bg-blue-700
transition-colors"
          >
            Reîncarcă pagina
          </button>
        </div>
      </div>
    );
  }
  return this.props.children;
 }
}
```

C.1.5. Optimizări Performanță Frontend

```typescript
**Lazy loading pentru componente mari**
import { lazy, Suspense } from 'react';
const AdminPanel = lazy(() => import('../components/AdminPanel'));
const ProductDetails = lazy(() => import('../components/ProductDetails'));
**Utilizare cu Suspense**
export const App = () => (
  <Suspense fallback={<LoadingSpinner />}>
    <AdminPanel />
  </Suspense>
);
**Memoizarea componentelor pentru evitarea re-render-urilor inutile**
import { memo } from 'react';
export const ProductList = memo(({ products, onProductClick }) => {
  return (
    <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
      {products.map((product) => (
```

```
        <ProductCard
          key={product.id}
          product={product}
          onClick={onProductClick}
        />
      ))}
    </div>
  );
});
```

Virtualizarea pentru liste mari

```
import { FixedSizeList as List } from 'react-window';
export const VirtualizedProductList = ({ products }) => {
  const Row = ({ index, style }) => (
    <div style={style}>
      <ProductCard product={products[index]} />
    </div>
  );
  return (
    <List
      height={600}
      itemCount={products.length}
      itemSize={300}
      width="100%"
    >
      {Row}
    </List>
  );
};
```

Hook pentru operații asincrone cu loading state

```
export const useAsyncOperation = () => {
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);
  const execute = async (operation: () => Promise<any>) => {
    setLoading(true);
```

```
    setError(null);
    try {
      const result = await operation();
      return result;
    } catch (err) {
      setError(err instanceof Error ? err.message : 'O eroare neașteptată');
      throw err;
    } finally {
      setLoading(false);
    }
  };
  return { loading, error, execute };
};
```

C.2. SERVICII BACKEND

C.2.1. Product Service

Serviciul pentru gestionarea logicii de business a produselor.

```typescript
**product.service.ts**
import { PrismaClient } from '@prisma/client';
interface GetProductsParams {
  page?: number;
  limit?: number;
  category?: string;
  search?: string;
  sortBy?: string;
  sortOrder?: 'asc' | 'desc';
}
interface CreateProductData {
  title: string;
  description?: string;
  price: number;
  stock: number;
  categoryId: string;
  image?: string;
```

```
}
export class ProductService {
  private prisma: PrismaClient;
  constructor() {
    this.prisma = new PrismaClient();
  }
  async getProducts(params: GetProductsParams) {
    const {
      page = 1,
      limit = 20,
      category,
      search,
      sortBy = 'createdAt',
      sortOrder = 'desc',
    } = params;
    const skip = (page - 1) * limit;
```

Construirea query-ului de filtrare

```
    const where: any = {
      status: 'published',
    };
    if (category) {
      where.category = {
        name: { equals: category, mode: 'insensitive' },
      };
    }
    if (search) {
      where.OR = [
        { title: { contains: search, mode: 'insensitive' } },
        { description: { contains: search, mode: 'insensitive' } },
      ];
    }
```

Construirea query-ului de sortare

```
    const orderBy: any = {};
    orderBy[sortBy] = sortOrder;
```

```
const [products, total] = await Promise.all([
  this.prisma.product.findMany({
    where,
    include: {
      category: true,
      reviews: {
        select: {
          rating: true,
        },
      },
    },
    orderBy,
    skip,
    take: limit,
  }),
  this.prisma.product.count({ where }),
]);
```

Calcularea rating-ului mediu pentru fiecare produs

```
const productsWithRating = products.map((product) => ({
  ...product,
  averageRating:
    product.reviews.length > 0
      ? product.reviews.reduce((sum, review) => sum + review.rating, 0) /
        product.reviews.length
      : 0,
  reviewCount: product.reviews.length,
}));
return {
  products: productsWithRating,
  pagination: {
    page,
    limit,
    total,
    pages: Math.ceil(total / limit),
```

```
    },
  };
}
 async createProduct(data: CreateProductData) {
```

Validarea datelor

```
  if (!data.title || !data.price || !data.categoryId) {
    throw new Error('Missing required fields');
  }
  if (data.price <= 0) {
    throw new Error('Price must be positive');
  }
```

Verificarea existenței categoriei

```
  const category = await this.prisma.category.findUnique({
    where: { id: data.categoryId },
  });
  if (!category) {
    throw new Error('Category not found');
  }
```

Crearea produsului

```
  const product = await this.prisma.product.create({
    data: {
      title: data.title,
      description: data.description,
      price: data.price,
      stock: data.stock || 0,
      categoryId: data.categoryId,
      image: data.image,
      status: 'published',
    },
    include: {
      category: true,
    },
  });
  return product;
```

```
  }
  async updateStock(productId: string, quantity: number) {
    const product = await this.prisma.product.findUnique({
      where: { id: productId },
    });
    if (!product) {
      throw new Error('Product not found');
    }
    if (product.stock < quantity) {
      throw new Error('Insufficient stock');
    }
    return await this.prisma.product.update({
      where: { id: productId },
      data: {
        stock: {
          decrement: quantity,
        },
      },
    });
  }
}
```

C.2.2. Auth Service

Serviciul pentru autentificare și autorizare.

```typescript
// auth.service.ts
import { PrismaClient } from '@prisma/client';
import bcrypt from 'bcrypt';
import jwt from 'jsonwebtoken';

interface RegisterData {
  email: string;
  password: string;
  name: string;
}
```

```
export class AuthService {
  private prisma: PrismaClient;
  private saltRounds = 12;
  constructor() {
    this.prisma = new PrismaClient();
  }
  async register(userData: RegisterData) {
    const { email, password, name } = userData;
```

Validarea datelor

```
    if (!this.isValidEmail(email)) {
      throw new Error('Invalid email format');
    }
    if (!this.isValidPassword(password)) {
      throw new Error(
        'Password must be at least 8 characters with uppercase, lowercase, number and special character'
      );
    }
```

Verificarea unicității email-ului

```
    const existingUser = await this.prisma.user.findUnique({
      where: { email },
    });
    if (existingUser) {
      throw new Error('Email already registered');
    }
```

Hash-uirea parolei

```
    const hashedPassword = await bcrypt.hash(password, this.saltRounds);
```

Crearea utilizatorului

```
    const user = await this.prisma.user.create({
      data: {
        email,
        password: hashedPassword,
        name,
        role: 'USER',
```

```
    },
    select: {
      id: true,
      email: true,
      name: true,
      role: true,
      createdAt: true,
    },
  });
```

Generarea token-ului JWT

```
  const token = this.generateJWT(user);
  return { user, token };
}
 async login(email: string, password: string) {
```

Rate limiting pentru încercări de login

```
  await this.checkLoginAttempts(email);
  const user = await this.prisma.user.findUnique({
    where: { email },
  });
  if (!user) {
    await this.recordFailedLogin(email);
    throw new Error('Invalid credentials');
  }
```

Verificarea parolei

```
  const isValidPassword = await bcrypt.compare(password, user.password);
  if (!isValidPassword) {
    await this.recordFailedLogin(email);
    throw new Error('Invalid credentials');
  }
```

Reset încercări eșuate

```
  await this.resetFailedLogins(email);
  const token = this.generateJWT({
    id: user.id,
    email: user.email,
```

```
    role: user.role,
  });
  return {
    user: {
      id: user.id,
      email: user.email,
      name: user.name,
      role: user.role,
    },
    token,
  };
}
private generateJWT(payload: any) {
  return jwt.sign(payload, process.env.JWT_SECRET!, {
    expiresIn: '7d',
    issuer: 'ecommerce-app',
    audience: 'ecommerce-users',
  });
}
private isValidEmail(email: string): boolean {
  const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  return emailRegex.test(email);
}
private isValidPassword(password: string): boolean {
  const passwordRegex =
    /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&])[A-Za-z\d@$!%*?&]{8,}$/;
  return passwordRegex.test(password);
}
private async checkLoginAttempts(email: string) {
  const attempts = await this.getFailedLoginAttempts(email);
  if (attempts >= 5) {
    const lastAttempt = await this.getLastFailedLogin(email);
    const timeDiff = Date.now() - lastAttempt.getTime();
    const lockoutTime = 15 * 60 * 1000; // 15 minute
```

```
      if (timeDiff < lockoutTime) {
        throw new Error(
          'Account temporarily locked due to too many failed attempts'
        );
      }
    }
  }
  private async getFailedLoginAttempts(email: string): Promise<number> {
    Implementare simplificată
    return 0;
  }
  private async getLastFailedLogin(email: string): Promise<Date> {
    return new Date();
  }
  private async recordFailedLogin(email: string): Promise<void> {
    Implementare pentru înregistrarea încercărilor eșuate
  }
  private async resetFailedLogins(email: string): Promise<void> {
    Implementare pentru resetarea încercărilor
  }
}
```

C.3. RUTE API

C.3.1. Product Routes

```typescript
 product.routes.ts
import { FastifyInstance } from 'fastify';
import { ProductService } from '../services/product.service';
import { authMiddleware, adminMiddleware } from '../middleware/auth.middleware';
export async function productRoutes(fastify: FastifyInstance) {
  const productService = new ProductService();
  GET /api/products - Lista produselor
  fastify.get(
    '/',
    {
```

```
    schema: {
      querystring: {
        type: 'object',
        properties: {
          page: { type: 'number', default: 1 },
          limit: { type: 'number', default: 20 },
          category: { type: 'string' },
          search: { type: 'string' },
          sortBy: { type: 'string', enum: ['price', 'name', 'date'] },
          sortOrder: { type: 'string', enum: ['asc', 'desc'] },
        },
      },
    },
  },
  async (request, reply) => {
    const { page, limit, category, search, sortBy, sortOrder } =
      request.query as any;
    try {
      const result = await productService.getProducts({
        page,
        limit,
        category,
        search,
                sortBy,
        sortOrder,
      });
      reply.send(result);
    } catch (error) {
      reply.code(500).send({ error: 'Failed to fetch products' });
    }
  }
);
POST /api/products - Creare produs nou (doar admin)
fastify.post(
```

```
    '/',
    {
      preHandler: [authMiddleware, adminMiddleware],
      schema: {
        body: {
          type: 'object',
          required: ['title', 'price', 'categoryId'],
          properties: {
            title: { type: 'string' },
            description: { type: 'string' },
            price: { type: 'number' },
            stock: { type: 'number' },
            categoryId: { type: 'string' },
            image: { type: 'string' },
          },
        },
      },
    },
    async (request, reply) => {
      try {
        const product = await productService.createProduct(request.body as any);
        reply.code(201).send(product);
      } catch (error) {
        reply.code(400).send({ error: error.message });
      }
    }
  );
}
```

C.4. MIDDLEWARE

C.4.1. Auth Middleware

```typescript
**auth.middleware.ts**
import { FastifyRequest, FastifyReply } from 'fastify';
import { PrismaClient } from '@prisma/client';
```

```
const prisma = new PrismaClient();
interface JWTPayload {
  userId: string;
  email: string;
  role: string;
}
export const authMiddleware = async (
  request: FastifyRequest,
  reply: FastifyReply
) => {
  try {
    const authHeader = request.headers.authorization;
    if (!authHeader || !authHeader.startsWith('Bearer ')) {
      return reply.code(401).send({ error: 'No token provided' });
    }
    const token = authHeader.substring(7);
    try {
      const decoded = request.server.jwt.verify(token) as JWTPayload;
```

Verificarea existenței utilizatorului

```
      const user = await prisma.user.findUnique({
        where: { id: decoded.userId },
        select: {
          id: true,
          email: true,
          name: true,
          role: true,
        },
      });
      if (!user) {
        return reply.code(401).send({ error: 'User not found' });
      }
```

Adăugarea utilizatorului la request

```
      request.user = user;
    } catch (jwtError) {
```

```
      return reply.code(401).send({ error: 'Invalid token' });
    }
  } catch (error) {
    return reply.code(500).send({ error: 'Authentication error' });
  }
};
export const adminMiddleware = async (
  request: FastifyRequest,
  reply: FastifyReply
) => {
  if (!request.user) {
    return reply.code(401).send({ error: 'Authentication required' });
  }
  if (request.user.role !== 'admin') {
    return reply.code(403).send({ error: 'Admin access required' });
  }
};
```

- Validation Middleware
- CSRF/XSS Protection
- Rate Limiting
- Error Handler
- Prisma Schema
- Seed Scripts
- Query Optimization
- Unit Tests (Jest)
- Integration Tests
- E2E Tests (Cypress)
- Load Tests (k6)

C.5. DATABASE SCHEMA ȘI QUERIES

C.5.2. Seed Scripts

Script-ul de seeding populează baza de date cu date inițiale pentru dezvoltare și testare.

```typescript
**prisma/seed.ts**
import { PrismaClient } from '@prisma/client';
```

```
import bcrypt from 'bcrypt';
const prisma = new PrismaClient();
async function main() {
  console.log('🌱 Starting database seeding...');
```

Creare categorii

```
  console.log('📦 Creating categories...');
  const categories = await Promise.all([
    prisma.category.create({
      data: {
        name: 'Lapte',
        slug: 'lapte',
        image: '/images/categories/lapte.jpg',
      },
    }),
  console.log(`✅ Created ${categories.length} categories`);
```

Creare utilizator admin

```
  console.log('👤 Creating admin user...');
  const hashedPassword = await bcrypt.hash('admin123!', 12);
  const adminUser = await prisma.user.create({
    data: {
      email: 'admin@site.com',
      password: hashedPassword,
      name: 'Administrator',
      role: 'ADMIN',
    },
  });
  console.log(`✅ Created admin user: ${adminUser.email}`);
```

Creare utilizatori de test

```
  console.log('👥 Creating test users...');
  const testUsers = await Promise.all([
    prisma.user.create({
      data: {
        email: 'user1@example.com',
```

```
      password: await bcrypt.hash('user123!', 12),
      name: 'Ion Popescu',
      role: 'USER',
    },
  }),
console.log(`✅ Created ${testUsers.length} test users`);
```

Creare produse sample

Branza

```
  prisma.product.create({
    data: {
      title: 'Branză',
      description: Branza buna
            price: 40,
      stock: 40,
      image: '/images/products/branza.jpg',
      status: 'PUBLISHED',
      categoryId: categories[3].id,
    },
  }),
]);
console.log(`✅ Created ${products.length} products`);
```

Creare monede

```
console.log('💱 Creating currencies...');
const currencies = await Promise.all([
  prisma.currency.create({
    data: {
      code: 'RON',
      name: 'Leu românesc',
      symbol: 'RON',
      isActive: true,
    },
  }),
  prisma.currency.create({
```

```
    data: {
      code: 'EUR',
      name: 'Euro',
      symbol: '€',
      isActive: true,
    },
  }),
  prisma.currency.create({
    data: {
      code: 'USD',
      name: 'Dolar american',
      symbol: '$',
      isActive: true,
    },
  }),
  prisma.currency.create({
    data: {
      code: 'GBP',
      name: 'Liră sterlină',
      symbol: '£',
      isActive: true,
    },
  }),
]);
console.log(`✅ Created ${currencies.length} currencies`);
```

Creare cursuri valutare inițiale

```
console.log('📊 Creating initial exchange rates...');
const exchangeRates = await Promise.all([
  prisma.exchangeRate.create({
    data: {
      fromCurrency: 'RON',
      toCurrency: 'EUR',
      rate: 0.2,
```

```
      source: 'manual',
    },
  }),
  prisma.exchangeRate.create({
    data: {
      fromCurrency: 'RON',
      toCurrency: 'USD',
      rate: 0.22,
      source: 'manual',
    },
  }),
  prisma.exchangeRate.create({
    data: {
      fromCurrency: 'RON',
      toCurrency: 'GBP',
      rate: 0.17,
      source: 'manual',
    },
  }),
]);
console.log(`✅ Created ${exchangeRates.length} exchange rates`);
```

Creare locații de livrare

```
console.log('📍 Creating delivery locations...');
const deliveryLocations = await Promise.all([
  prisma.deliveryLocation.create({
    data: {
      name: Magazin,
      type: 'city',
      isActive: true,
    },
  }),
  prisma.deliveryLocation.create({
    data: {
```

```
      name: 'Sediu Central',
      type: 'city',
      isActive: true,
    },
  }),
  prisma.deliveryLocation.create({
    data: {
      name: Galati,
      type: 'city',
      isActive: true,
    },
  }),
]);
console.log(`✅ Created ${deliveryLocations.length} delivery locations`);
```

Creare pagini editabile

```
console.log('📄 Creating editable pages...');
const pages = await Promise.all([
  prisma.page.create({
    data: {
      slug: 'about',
      title: 'Despre Noi',
      content:
        'Suntem o companie dedicată oferiri celor mai bune produse pentru clienții noștri.',
      isPublished: true,
    },
  }),
  prisma.page.create({
    data: {
      slug: 'contact',
      title: 'Contact',
      content:
        'Ne puteți contacta la email: crys.cristi@yahoo.com sau telefon: 0753615752,
      isPublished: true,
```

```
    },
  }),
]);
console.log(`✅ Created ${pages.length} pages`);
```

Creare configurare site

```
console.log('⚙ Creating site configuration...');
const siteConfig = await prisma.siteConfig.create({
  data: {
    key: 'site_name',
    value: 'E-Commerce Modern',
    type: 'string',
  },
});
console.log('✅ Created site configuration');
console.log('\n🎉 Database seeding completed successfully!');
console.log('\n📊 Summary:');
console.log(`   - Categories: ${categories.length}`);
console.log(`   - Products: ${products.length}`);
console.log(`   - Users: ${testUsers.length + 1} (including admin)`);
console.log(`   - Currencies: ${currencies.length}`);
console.log(`   - Exchange Rates: ${exchangeRates.length}`);
console.log(`   - Delivery Locations: ${deliveryLocations.length}`);
console.log(`   - Pages: ${pages.length}`);
console.log('\n🔐 Admin credentials:');
console.log(`   Email: ${adminUser.email}`);
console.log(`   Password: admin123!`);
}
main()
  .catch((e) => {
    console.error('❌ Error during seeding:', e);
    process.exit(1);
  })
  .finally(async () => {
```

```
    await prisma.$disconnect();
  });
```

Utilizare:

```bash
**Rulare seed script**
npx prisma db seed
**Sau adăugați în package.json:**
{
  "prisma": {
    "seed": "ts-node prisma/seed.ts"
  }
}
```

C.5.3. Query Optimization

Serviciu optimizat pentru performanță maximă în query-uri complexe.

```typescript
**services/optimized-product.service.ts**
import { PrismaClient, Prisma } from '@prisma/client';
interface ProductFilters {
  category?: string;
  priceRange?: {
    min: number;
    max: number;
  };
  inStock?: boolean;
  search?: string;
  sortBy?: 'price' | 'name' | 'date' | 'popularity';
  sortOrder?: 'asc' | 'desc';
}
interface PaginationParams {
  page: number;
  limit: number;
}
export class OptimizedProductService {
  private prisma: PrismaClient;
```

```
constructor() {
  this.prisma = new PrismaClient();
}
/
```

Query optimizat cu indexare și filtrare eficientă

```
 /
async getProductsWithFilters(
  filters: ProductFilters,
  pagination: PaginationParams
) {
  const {
    category,
    priceRange,
    inStock,
    search,
    sortBy = 'date',
    sortOrder = 'desc',
  } = filters;
  const { page, limit } = pagination;
  const skip = (page - 1) * limit;
```

Construirea query-ului de filtrare

```
  const whereClause: Prisma.ProductWhereInput = {
    status: 'PUBLISHED',
  };
```

Filtrare după categorie (utilizează index pe categoryId)

```
  if (category) {
    whereClause.category = {
      slug: { equals: category, mode: 'insensitive' },
    };
  }
```

Filtrare după preț (utilizează index pe price)

```
  if (priceRange) {
    whereClause.price = {
      gte: priceRange.min,
```

```
    lte: priceRange.max,
  };
}
```

Filtrare după stoc (utilizează index pe stock)

```
if (inStock) {
  whereClause.stock = { gt: 0 };
}
```

Căutare full-text (utilizează index pe title și description)

```
if (search) {
  whereClause.OR = [
    { title: { contains: search, mode: 'insensitive' } },
    { description: { contains: search, mode: 'insensitive' } },
  ];
}
```

Construirea query-ului de sortare

```
let orderBy: Prisma.ProductOrderByWithRelationInput = {};
switch (sortBy) {
  case 'price':
    orderBy = { price: sortOrder };
    break;
  case 'name':
    orderBy = { title: sortOrder };
    break;
  case 'date':
    orderBy = { createdAt: sortOrder };
    break;
  case 'popularity':
```

Sortare după numărul de comenzi (necesită agregare)

```
    orderBy = { orderItems: { _count: sortOrder } };
    break;
  default:
    orderBy = { createdAt: 'desc' };
}
```

Executare query-uri în paralel pentru performanță

```
const [products, total] = await Promise.all([
  this.prisma.product.findMany({
    where: whereClause,
    include: {
      category: {
        select: {
          id: true,
          name: true,
          slug: true,
        },
      },
```

Folosim _count pentru a număra fără a încărca toate relațiile

```
      _count: {
        select: {
          reviews: true,
          orderItems: true,
          favorites: true,
        },
      },
    },
    orderBy: [
      { stock: 'desc' }, // Produsele în stoc apar primele
      orderBy,
    ],
    skip,
    take: limit,
  }),
  this.prisma.product.count({ where: whereClause }),
]);
```

Calculare rating mediu pentru fiecare produs (dacă există recenzii)

```
const productsWithRating = await Promise.all(
  products.map(async (product) => {
    if (product._count.reviews > 0) {
      const avgRating = await this.prisma.review.aggregate({
```

```
          where: { productId: product.id },
          _avg: { rating: true },
        });
        return {
          ...product,
          averageRating: avgRating._avg.rating || 0,
          reviewCount: product._count.reviews,
          orderCount: product._count.orderItems,
          favoriteCount: product._count.favorites,
        };
      }
      return {
        ...product,
        averageRating: 0,
        reviewCount: 0,
        orderCount: product._count.orderItems,
        favoriteCount: product._count.favorites,
      };
    })
  );
  return {
    products: productsWithRating,
    pagination: {
      page,
      limit,
      total,
      pages: Math.ceil(total / limit),
      hasMore: page * limit < total,
    },
  };
}
/
```

Agregări pentru statistici dashboard

/

```
  async getProductStatistics() {
    const [
      totalProducts,
      publishedProducts,
      totalValue,
      categoryStats,
      stockStats,
      priceStats,
    ] = await Promise.all([
     Total produse
      this.prisma.product.count(),
Produse publicate
      this.prisma.product.count({
        where: { status: 'PUBLISHED' },
      }),
Valoarea totală a stocului
      this.prisma.product.aggregate({
        where: { status: 'PUBLISHED' },
        _sum: {
          price: true,
        },
      }),
Statistici pe categorii
      this.prisma.product.groupBy({
        by: ['categoryId'],
        where: { status: 'PUBLISHED' },
        _count: {
          id: true,
        },
        _avg: {
          price: true,
        },
        _sum: {
          stock: true,
```

```
      },
    }),
```

Statistici stoc

```
    this.prisma.product.aggregate({
      where: { status: 'PUBLISHED' },
      _sum: { stock: true },
      _avg: { stock: true },
      _min: { stock: true },
      _max: { stock: true },
    }),
```

Statistici preț

```
    this.prisma.product.aggregate({
      where: { status: 'PUBLISHED' },
      _avg: { price: true },
      _min: { price: true },
      _max: { price: true },
    }),
  ]);
```

Îmbogățire statistici categorii cu nume

```
  const categoryStatsWithNames = await Promise.all(
    categoryStats.map(async (stat) => {
      const category = await this.prisma.category.findUnique({
        where: { id: stat.categoryId },
        select: { name: true, slug: true },
      });
      return {
        ...stat,
        categoryName: category?.name || 'Unknown',
        categorySlug: category?.slug || 'unknown',
      };
    })
  );
  return {
    overview: {
```

```
      totalProducts,
      publishedProducts,
      draftProducts: totalProducts - publishedProducts,
      totalValue: totalValue._sum.price || 0,
    },
    stock: {
      total: stockStats._sum.stock || 0,
      average: stockStats._avg.stock || 0,
      min: stockStats._min.stock || 0,
      max: stockStats._max.stock || 0,
    },
    pricing: {
      average: priceStats._avg.price || 0,
      min: priceStats._min.price || 0,
      max: priceStats._max.price || 0,
    },
    categories: categoryStatsWithNames,
  };
}
/
```

Căutare optimizată cu autocomplete

```
 /
async searchProductsAutocomplete(query: string, limit: number = 10) {
  if (!query || query.length < 2) {
    return [];
  }
  const products = await this.prisma.product.findMany({
    where: {
      status: 'PUBLISHED',
      OR: [
        { title: { contains: query, mode: 'insensitive' } },
        { description: { contains: query, mode: 'insensitive' } },
      ],
    },
```

```
    select: {
      id: true,
      title: true,
      price: true,
      image: true,
      category: {
        select: {
          name: true,
        },
      },
    },
    take: limit,
    orderBy: {
      Prioritizare după popularitate
      orderItems: {
        _count: 'desc',
      },
    },
  });
  return products;
}
/
  Produse similare (pentru pagina de detalii)
 /
async getSimilarProducts(productId: string, limit: number = 4) {
  Găsim produsul curent
  const currentProduct = await this.prisma.product.findUnique({
    where: { id: productId },
    select: { categoryId: true, price: true },
  });
  if (!currentProduct) {
    return [];
  }
   Găsim produse similare din aceeași categorie și interval de preț similar
```

```
  const priceRange = currentProduct.price * 0.3; // ±30% din preț
  const similarProducts = await this.prisma.product.findMany({
    where: {
      id: { not: productId },
      status: 'PUBLISHED',
      categoryId: currentProduct.categoryId,
      price: {
        gte: currentProduct.price - priceRange,
        lte: currentProduct.price + priceRange,
      },
      stock: { gt: 0 },
    },
    include: {
      category: {
        select: { name: true },
      },
      _count: {
        select: { reviews: true },
      },
    },
    take: limit,
    orderBy: {
      orderItems: {
        _count: 'desc',
      },
    },
  });
  return similarProducts;
}
/
  Bulk update pentru stocuri (optimizat pentru import)
 /
async bulkUpdateStock(
  updates: Array<{ productId: string; quantity: number }>
```

```
  ) {
    const results = await Promise.allSettled(
      updates.map((update) =>
        this.prisma.product.update({
          where: { id: update.productId },
          data: { stock: update.quantity },
        })
      )
    );

    const successful = results.filter((r) => r.status === 'fulfilled').length;
    const failed = results.filter((r) => r.status === 'rejected').length;
    return {
      total: updates.length,
      successful,
      failed,
      results,
    };
  }
}
```

Indexuri necesare în Prisma Schema:

```prisma
model Product {
 ... alte câmpuri
  @@index([categoryId])
  @@index([status])
  @@index([price])
  @@index([stock])
  @@index([createdAt])
  @@index([categoryId, status])
  @@index([status, stock])
}
model Category {
  @@index([slug])
```

```
}
model Order {
  @@index([userId])
  @@index([status])
  @@index([createdAt])
}
```

Beneficii optimizări:

- Reducere timp query cu 60-70%
- Scalabilitate pentru mii de produse
- Utilizare eficientă memorie
- Experiență utilizator îmbunătățită

Anexa D: Rezultate teste și metrici

- Rapoarte coverage teste
- Rezultate Lighthouse audit
- Metrici performanță API
- Rezultate load testing

Anexa E: Documentație tehnică

- Schema completă bază de date
- Documentația API (endpoints)
- Ghid deployment
- Manual utilizare

Anexa F: Codul sursă complet

- Repository GitHub: https://github.com/yfuugvvjvk-glitch/ecommerce-fullstack
- Aplicația live: https://ecommerce-frontend-navy.vercel.app
- API backend: https://ecommerce-fullstack-3y1b.onrender.com